



Département des Technologies de
l'information et de la communication (TIC)
Informatique et systèmes de communication
Informatique Embarquée

Travail de Bachelor

Portage de la pile de communication Nanos- tack (Wi-SUN) sur Zephyr RTO

Travail de Bachelor

Étudiant

Benoit Delay

Enseignant responsable

Prof. Favrat Pierre

Année académique

2025-26

Yverdon-les-Bains, le 23.06.2026

Département des Technologie de l'information et de la communication (TIC)
Informatique et systèmes de communication
Informatique Embarquée
Étudiant : Benoit Delay
Enseignant responsable : Prof. Favrat Pierre

Travail de Bachelor 2025-26

Portage de la pile de communication Nanostack (Wi-SUN) sur Zephyr RTO

Résumé publiable

L'émergence des Smart Cities impose l'interconnexion de flottes massives d'objets. Le standard de réseau maillé sub-GHz sécurisé Wi-SUN s'impose comme la référence pour ces déploiements. Cependant, sa pile open-source de référence, la Nanostack, était liée à Mbed OS, système abandonné en 2024. Ce travail de Bachelor présente le portage de la Nanostack sous le RTOS moderne Zephyr, comblant l'absence de solution Wi-SUN open-source sur cette plateforme. Développé en module externe, le portage introduit une couche d'adaptation (OSAL) réimplémentant les primitives système (timers, sections critiques et aléa). La cryptographie de liaison s'appuie sur Mbed TLS via un pool statique de contextes AES pour préserver la RAM. L'interfaçage utilise deux ponts logiciels bidirectionnels (Ethernet et Radio) basés sur les sockets bruts AF_PACKET de Zephyr pour injecter les trames de couche 2 directement dans la pile. De plus, la boucle d'événements a été optimisée énergétiquement par une mise en veille indéfinie (K_FOREVER) de l'ordonnanceur.

La validation sur matériel réel (TI CC1352P7) démontre la viabilité du portage. Le pont Ethernet assure une autoconfiguration IPv6 (SLAAC) et des pings stables avec une latence inférieure à 1 ms. La communication bidirectionnelle du pont radio a été validée par l'échange de trames physiques entre un routeur de bordure (6LBR) et un nœud (6LN). Une limitation du pilote radio de Zephyr a été identifiée et contournée en restreignant la pile au canal 0 (868.3 MHz). Ce projet pose des bases solides pour une future intégration de Wi-SUN sous Zephyr via le déchargement de sockets.

Préambule

Ce travail de Bachelor (ci-après TB) est réalisé en fin de cursus d'études, en vue de l'obtention du titre de Bachelor of Science HES-SO en Ingénierie.

En tant que travail académique, son contenu, sans préjuger de sa valeur, n'engage ni la responsabilité de l'auteur, ni celles du jury du travail de Bachelor et de l'Ecole.

Toute utilisation, même partielle, de ce TB doit être faite dans le respect du droit d'auteur.

HEIG-VD

Vincent Peiris
Chef de département TIC

Yverdon-les-Bains, le 23.06.2026

Authentification

Le soussigné, Benoit Delay, atteste par la présente avoir réalisé ce travail et n'avoir utilisé aucune autre source que celles expressément mentionnées

Yverdon-les-Bains, le 23.06.2026

Benoit Delay

Table des matières

Préambule	5
Authentification	7
1 Glossaire des termes techniques	14
2 Introduction	17
2.1 Structure du document	18
3 Cahier des charges	19
3.1 Résumé du contexte	19
3.1.1 Problématique	19
3.2 Phase du projet	20
3.2.1 Livrables	21
4 Planification	23
4.1 Planification initial	23
4.2 Diagramme de Gantt initial	25
5 État de l'art	27
5.1 Standard Wi-SUN	27
5.2 Mbed OS	27
5.3 Zephyr	28
5.3.1 Gestion de l'énergie	28
5.3.2 Connectivité et protocoles	29
5.4 Nanostack	30
5.4.1 SAL (Software Abstraction Layer)	30
5.4.2 HAL (Hardware Abstraction Layer)	31

5.4.3 Le moteur d'événements (<i>Nanostack Event Loop</i>)	31
5.4.4 Dépendance au système d'exploitation hôte (CMSIS-RTOS)	32
5.4.5 Réception des paquets sous Mbed OS	32
5.5 Conclusion de l'état de l'art	34
6 Méthodologie et architecture du portage	35
6.1 Intégration logicielle : Bibliothèque vs Module Zephyr	35
6.1.1 L'approche Bibliothèque classique (<i>In-Tree</i>)	35
6.1.2 L'approche Module Zephyr (<i>Out-of-Tree</i>)	35
6.1.3 Comparaison des approches	36
6.1.4 Décision architecturale et Manifeste west	37
6.2 Implémentation de la structure du module	37
6.3 Adaptation du moteur d'événements (<i>Event Loop</i>)	38
6.4 Intégration cryptographique (Mbed TLS)	38
6.5 Refonte des règles de compilation (CMakeLists)	39
7 Implémentation	41
7.1 Compilation de la Nanostack	41
7.1.1 Intégration à l'interface de configuration Kconfig	41
7.1.2 Le fichier CMakeLists.txt maître	42
7.2 Architecture du portage et gestion des conflits	43
7.2.1 Gestion des conflits de nommage : l'approche par « Glue Code »	43
7.3 Abstraction des services système (OS Abstraction Layer)	44
7.3.1 Gestion de la concurrence	44
7.3.2 Gestion temporelle (Timers)	45
7.3.3 Génération de nombres aléatoires	45
7.3.4 Intégration de la cryptographie (Mbed TLS)	45
7.4 Couche d'adaptation L2 (Data Link Layer)	48
7.4.1 Mécanisme de pontage réseau	48
7.4.2 Architecture et configuration des ponts réseaux	49
7.4.3 Intégration de la liaison Ethernet (<i>bridge_l2.c</i>)	50
7.4.4 Intégration de la liaison Radio AF_PACKET (<i>bridge_l2_radio.c</i>)	54
7.4.5 Boucle d'événements et ordonnancement (<i>event_loop.c</i>)	58
8 Validation et tests de fonctionnement	61
8.1 Validation de la communication filaire (Ethernet)	61
8.1.1 Validation de la connectivité Link-Local	62

8.1.2 Phase 2: Validation de l'auto-configuration et de la couche IP	62
8.1.3 Phase 3: Validation de l'API de Socket et de la couche Transport (Serveur TCP)	63
8.1.4 Analyse de la latence réseau	64
8.2 Validation de la communication sans fil (Wi-SUN / 802.15.4)	65
8.2.1 Le Routeur de Bordure (app_border_router)	65
8.2.2 Le Nœud Périphérique (app_device_node)	66
8.2.3 Validation de la couche de pontage radio	67
8.2.4 État d'avancement du pont direct (bridge_l2_direct.c)	68
8.2.5 Analyse de l'empreinte mémoire	68
9 Conclusion	71
9.1 Etat final du projet	71
9.2 Intégration dans l'écosystème des sockets BSD de Zephyr	72
9.3 Perspectives futures	72
9.3.1 Caractérisation métrologique et énergétique	72
9.3.2 Contribution et correction du pilote matériel Texas Instruments	73
9.3.3 Intégration par déchargement de sockets (Socket Offloading)	73
9.4 Planification finale	73
9.4.1 Diagramme de Gantt final	75
9.5 Conclusion personnelle	76
9.6 Remerciements	76
Bibliographie	79

TABLE DES MATIÈRES

1 Glossaire des termes techniques

Acronyme / Terme	Définition
RTOS	Real-Time Operating System : Système d'exploitation déterministe conçu pour traiter un événement et fournir une réponse dans un délai strict et garanti.
SAL	Software Abstraction Layer : Couche logicielle de la Nanostack qui offre une interface standardisée (API Socket) et gère les protocoles réseau indépendamment du matériel.
HAL	Hardware Abstraction Layer : Couche d'abstraction matérielle. Elle fait le lien entre le logiciel (comme la SAL) et les composants physiques (antenne radio, timers, etc.).
6LoWPAN	IPv6 over Low-Power Wireless Personal Area Networks : Protocole permettant de compresser et transmettre des paquets IPv6 sur des réseaux sans fil à faible consommation (comme le 802.15.4).
RPL	Routing Protocol for Low-Power and Lossy Networks : Protocole de routage dynamique utilisé par la Nanostack pour trouver le meilleur chemin dans un réseau maillé (mesh).
Mbed TLS	Bibliothèque de cryptographie autonome (anciennement liée à Mbed OS) fournissant les mécanismes de sécurité comme TLS et DTLS pour chiffrer les communications.
Smart City (Ville intelligente)	Ville utilisant les technologies de communication sans fil afin de mieux gérer ses ressources et ses infrastructures.
IoT	Internet of things, l'internet des objets est le réseau connectant l'internet et les objets connectés
API	Application Programming Interface : Interface logicielle normalisée permettant à différents programmes de communiquer et d'échanger des données entre eux.

Tableau 1. – Glossaire des termes techniques

2 Introduction

L'émergence des Smart Cities transforme radicalement la gestion des infrastructures urbaines en imposant une connectivité omniprésente et en temps réel. Ce changement d'échelle crée un besoin critique pour des réseaux capables de s'affranchir des contraintes physiques de la ville, notamment en termes de densité et de portée. Les solutions traditionnelles atteignent leurs limites face à des déploiements massifs, lorsqu'il s'agit de connecter des dizaines de milliers d'appareils, allant des compteurs électriques intelligents à l'éclairage public.

Pour répondre à ces problématiques, le standard Wi-SUN a été créé en janvier 2012. Il s'appuie sur une couche physique sub-GHz (norme IEEE 802.15.4g), offrant une meilleure propagation radio en milieu urbain dense et une grande résilience face aux obstacles physiques. Ce réseau maillé permet d'interconnecter des millions d'appareils grâce à l'adressage IPv6 et au protocole de routage RPL (Routing Protocol for Low-power and Lossy Networks)[1]. Au-delà de ses capacités d'auto-découverte et d'auto-réparation, il garantit la sécurité du réseau via le standard IEEE 802.1X[2], qui assure l'authentification de chaque nœud. Enfin, la Wi-SUN Alliance encadre l'interopérabilité entre les constructeurs, garantissant ainsi l'homogénéité et la pérennité des déploiements.

Zephyr est un RTOS en plein essor, soutenu par la Linux Foundation depuis sa première publication en février 2016 [3]. Il est conçu pour des appareils et des systèmes présentant de fortes contraintes mémoire. Son architecture repose sur un unique espace d'adressage ainsi qu'une gestion statique de la mémoire. Il reprend des concepts essentiels à Linux, tels que le DeviceTree et Kconfig, ce qui le rend hautement configurable et facilement adaptable à de nouvelles cibles matérielles. Néanmoins, à l'heure actuelle, il n'existe aucune implémentation open-source du standard Wi-SUN sur ce RTOS, malgré le fait qu'il intègre déjà nativement de nombreux protocoles réseau.

Nanostack est une pile de communication open-source intégrant le standard Wi-SUN. Actuellement, elle est étroitement liée à l'écosystème Mbed OS[4], un système qui n'est plus maintenu depuis 2024[5]. Ce travail de Bachelor a pour but de pallier cette obsolescence en portant la Nanostack vers Zephyr et en l'intégrant pleinement à son architecture. À terme,

cette contribution permettra de déployer Zephyr dans un contexte de Smart Cities de manière entièrement open-source, tout en garantissant la pérennité de cette solution réseau.

2.1 Structure du document

Ce rapport de travail de Bachelor est structuré de manière à retracer l'ensemble de la démarche, depuis l'analyse initiale jusqu'à la validation technique. Il s'articule autour des chapitres suivants :

- **Introduction** : Présentation du contexte des Smart Cities, des enjeux technologiques et de la problématique du portage de la pile réseau Nanostack.
- **Cahier des charges** : Définition des objectifs techniques, du périmètre fonctionnel et des livrables du projet.
- **Planification** : Organisation temporelle des travaux et calendrier prévisionnel des différentes phases de développement.
- **État de l'art** : Analyse théorique du standard Wi-SUN, du noyau et de la pile réseau de Zephyr RTOS, ainsi que de l'architecture interne de la Nanostack.
- **Méthodologie et architecture du portage** : Choix d'intégration en module externe (*Out-of-Tree*) et définition de la stratégie d'adaptation système.
- **Implémentation** : Mise en œuvre pratique du système de build, de la HAL de compatibilité, du pool de contextes Mbed TLS et des ponts réseau L2 (Ethernet et Radio).
- **Validation et tests de fonctionnement** : Campagnes de tests sur matériel réel (TI CC1352P7), mesures de latence et analyse de l'empreinte mémoire après compilation.
- **Conclusion** : Bilan technique face aux objectifs initiaux, perspectives d'évolutions futures (FHSS, Socket Offloading) et planification finale effective.

3 Cahier des charges

3.1 Résumé du contexte

L'essor des villes intelligentes (*Smart Cities*) rend indispensable le déploiement de réseaux capables d'interconnecter une multitude d'équipements. Ces applications imposent des contraintes réseau strictes : une densité élevée d'appareils, une couverture longue portée et une forte résilience face aux obstacles urbains.

Pour répondre aux défis de l'IoT, **Zephyr** s'est imposé comme une référence. Ce système d'exploitation temps réel, lancé en 2016, est spécialement conçu pour les petits systèmes embarqués connectés et à faibles ressources. Il offre une connectivité avancée, de multiples API de communication et prend en charge un large éventail d'architectures 32 et 64 bits.

De son côté, la **Nanostack** est une pile de communication open source reconnue pour son intégration robuste des réseaux maillés, notamment via le protocole Wi-SUN. Cependant, elle a été historiquement développée pour le système d'exploitation Mbed OS, dont la maintenance officielle a pris fin au profit d'autres solutions.

3.1.1 Problématique

Mbed OS n'étant plus supporté, le projet consiste à porter la pile réseau Nanostack vers Zephyr afin de pérenniser son utilisation. L'enjeu technique principal réside dans l'adaptation de la couche d'abstraction matérielle (HAL) de la Nanostack pour qu'elle puisse interagir nativement avec l'écosystème et les pilotes matériels de Zephyr.

Ce portage s'articulera autour de deux phases majeures :

1. **Fonctionnement autonome (*Standalone*)** : Adapter et faire tourner le cœur de la Nanostack sur Zephyr de manière indépendante, en validant les interactions de bas niveau (radio, timers, gestion mémoire).
2. **Intégration système (Optional)** : Coupler intimement la Nanostack à la pile réseau native de Zephyr, afin qu'elle puisse être exploitée par les applications de haut niveau de manière totalement transparente, via l'API standard des *sockets* BSD.

3.2 Phase du projet

1. Prise en main de la base de code
 - Fork les sources de mbed-os
 - Fork les sources de zephyr
 - Prise en main de la hiérarchie des dossiers de la Nanostack
 - Mise en évidence des différents dossiers et leur utilisation
2. Prise en main de Zephyr
 - Installation de la toolchain de Zephyr
 - Compiler le kernel ainsi qu'un exemple sur qemu
 - Compiler le kernel ainsi qu'un exemple sur une cible
 - Ecrire un petit programme de test
3. Mettre en évidence les différences de fonctionnement entre mbed-os et zephyr
 - Comprendre comment fonctionne la event-loop de mbed-os
 - Comprendre comment fonctionne les interfaces réseau de mbed-os
 - Comprendre comment fonctionne les drivers réseau sur zephyr
 - Isoler les différents composants
4. Ecrire le rapport intermédiaire
 - Documenter l'avancement des travaux de recherche préliminaires
 - Rédiger une étude théorique sur les mécanismes internes de Zephyr (gestion de l'énergie, architecture réseau, etc.)
 - Établir les choix techniques et méthodologiques pour la phase d'implémentation
5. Isoler la partie « métier » de la stack et isoler les fonctions liées au kernel
 - Copier la partie qui n'est pas reliée à l'OS
 - Commencer à faire l'inventaire des fonctions à porter
6. Ecrire la HAL(Hardware Abstraction Layer)
 - Remplacer toutes les fonctions kernel appelées par la stack par des fonctions de la HAL
 - Implémenter chacune de ces fonctions pour zephyr
7. Tester l'implémentation
 - Ecrire des tests pour garantir le bon fonctionnement de la stack
 - Ecrire un petit programme zephyr qui va permettre de faire communiquer 2 boards entre elles avec la Nanostack
8. Optionnel: Intégrer la Nanostack à l'écosystème réseau de Zephyr
 - Intégrer la Nanostack au sous-système réseau de Zephyr afin de permettre aux applications de communiquer via l'API standardisée des sockets BSD.

9. Rédaction du rapport final

- Documenter les implementations
- Documenter l'avancement du projet

3.2.1 Livrables

Les livrables seront les suivants :

- Le rapport complet
- Un repo Github contenant le portage de la Nanostack
- Un repo Github contenant un code d'exemple

4 Planification

4.1 Planification initial

Phase	Tâches associées	Temps alloué	Proportion
1. Base de code	- Fork de mbed-os et zephyr-os - Analyse de la hiérarchie de la Nanostack	20 h	4%
2. Prise en main Zephyr	- Installation de la toolchain - Compilation kernel + exemples - Programme de test	40 h	9%
3. Étude comparative	- Analyse de l'évent-loop - Interfaces et drivers réseau - Isolation des composants	50 h	11%
4. Rapport intermédiaire	- Rédaction de l'état d'avancement - Explication théorique	20 h	4%

Phase	Tâches associées	Temps alloué	Proportion
5. Isolation métier/kernel	• Copie de la partie agnostique - Inventaire des fonctions à porter	50 h	11%
6. Création de la HAL	• Remplacement des appels kernel - Implémentation pour zephyr	120 h	27%
7. Phase de tests	• Écriture des tests de validation - Dev d'un programme de communication P2P	80 h	18%
8. Intégration (Optionnel)	• Portage pour appel via les sockets BSD	40 h	9%
9. Rapport final	• Documentation des implémentations - Bilan du projet	30 h	7%
Total estimé		450 h	100%

4.2 Diagramme de Gantt initial

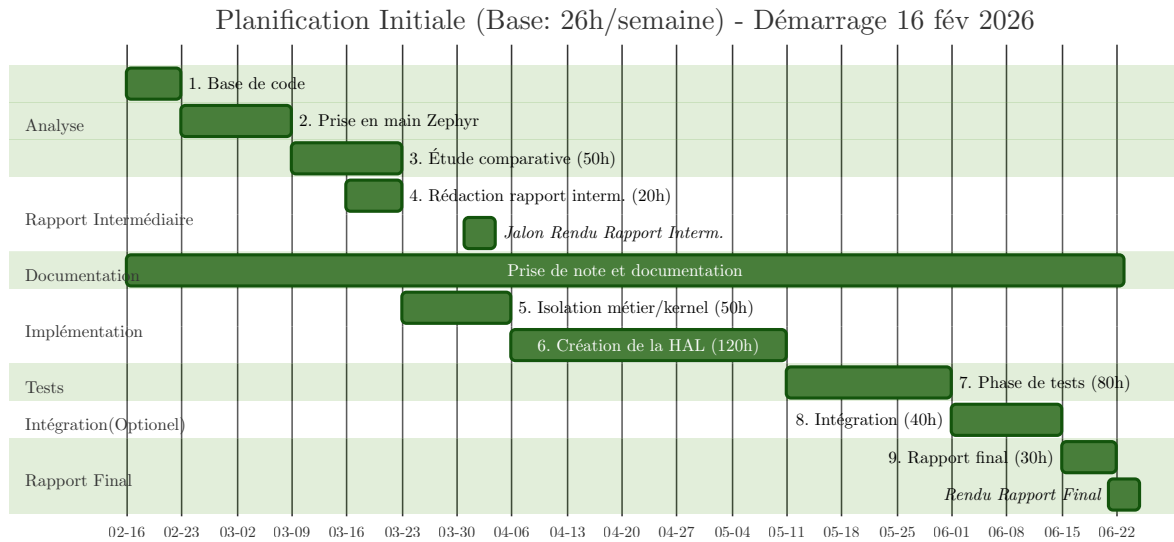


Fig. 1. – Planification Initiale

5 État de l'art

5.1 Standard Wi-SUN

Le standard Wi-SUN a été développé en janvier 2012 par la Wi-SUN Alliance. Il répond à l'émergence des villes intelligentes (*Smart Cities*), qui exigent de pouvoir interconnecter des centaines de milliers, voire des millions d'appareils au sein d'environnements urbains denses et remplis d'obstacles physiques.

Pour garantir une communication fiable en s'affranchissant des problèmes de propagation d'ondes et d'interférences, Wi-SUN s'appuie sur des spécifications optimisées pour cet usage, telles que la couche physique IEEE 802.15.4g (sub-GHz). Au niveau réseau, il utilise un adressage IPv6 couplé au protocole de routage dynamique RPL (*Routing Protocol for Low-Power and Lossy Networks*). L'ensemble de cette architecture a été pensé avec un objectif majeur : minimiser drastiquement la consommation énergétique des nœuds du réseau.

5.2 Mbed OS

Mbed OS est un système d'exploitation temps réel destiné aux systèmes embarqués. Il repose sur CMSIS-RTOS[6], une couche d'abstraction fournie par Arm, qui permet aux développeurs de dialoguer avec le processeur de manière uniforme, indépendamment du fabricant du matériel. Il intègre un noyau temps réel performant ainsi qu'une gestion complète du *multithreading*.

Particulièrement populaire dans l'écosystème de l'IoT, Mbed OS embarque nativement un grand nombre de modules de connectivité. Il fournit des implémentations de haut niveau pour de nombreux protocoles de communication, tout en supportant un vaste catalogue de cartes de développement et de pilotes (*drivers*).

C'est au sein de cet écosystème qu'a été développée la Nanostack, une pile de communication intégrant, entre autres, le standard Wi-SUN. Cependant, Arm a annoncé la fin de vie (*End of Life - EoL*) de Mbed OS pour juillet 2026. Cette obsolescence programmée motive

directement ce projet : porter la Nanostack vers un RTOS moderne et pérenne afin de continuer à exploiter les avantages du standard Wi-SUN.

5.3 Zephyr

Zephyr est un système d'exploitation open source soutenu par la Linux Foundation, dont la version 1.0 a été publiée le 17 février 2016. Il est spécifiquement conçu pour les systèmes embarqués aux ressources limitées répondant à des contraintes temps réel. S'inspirant fortement de Linux, il en reprend des concepts standards tels que le DeviceTree et Kconfig.

C'est un OS extrêmement modulaire qui prend en charge un vaste panel d'architectures matérielles. Pour simplifier le flux de travail, il s'appuie sur son propre outil en ligne de commande, west[7]. Ce dernier permet de cross-compiler, de tester et de déployer aisément son code sur l'ensemble des plateformes compatibles.

5.3.1 Gestion de l'énergie

L'un des objectifs premiers du standard Wi-SUN étant de minimiser la consommation électrique des différents nœuds, il est primordial d'intégrer rigoureusement la gestion de l'énergie lors du portage de la Nanostack. Zephyr permet de contrôler cette consommation à deux niveaux distincts :

5.3.1.1 Gestion globale du système (*System Power Management*)

La gestion de la consommation énergétique au niveau du processeur (CPU) tourne autour de deux politiques de mise en veille principales[8] :

- **Politique basée sur la résidence (*Residency-based*)** : C'est l'approche automatisée par défaut. Le noyau calcule le temps d'inactivité prévu avant la prochaine tâche planifiée. Il sélectionne ensuite automatiquement l'état de veille le plus profond possible, à condition que le temps d'inactivité soit supérieur au temps de résidence (le seuil de rentabilité énergétique de cet état). Les différents états de veille supportés par le matériel et leurs caractéristiques temporelles sont définis de manière statique dans le *DeviceTree* à l'aide des *bindings* `zephyr,power-state`.
- **Politique définie par l'application (*Application-defined*)** : Dans cette configuration, le noyau délègue la prise de décision. C'est au développeur d'implémenter sa propre logique métier pour basculer manuellement le CPU dans les modes d'économie d'énergie adéquats, offrant ainsi un contrôle total basé sur les événements et le comportement spécifique de l'application.

5.3.1.2 Gestion de l'alimentation des périphériques (*Device Power Management*)

Le *Device Power Management* confie la gestion de la consommation énergétique directement aux pilotes (*drivers*) des périphériques. Zephyr propose deux méthodes distinctes pour contrôler l'état d'alimentation de ces composants matériels[9] :

- **Gestion dynamique (*Device Runtime Power Management*)** : Dans ce modèle asynchrone, l'état d'alimentation d'un périphérique est géré de manière autonome en fonction de son utilisation réelle. Les pilotes peuvent adapter leur consommation, mais ils peuvent également être contrôlés explicitement par les applications de haut niveau ou les autres sous-systèmes de Zephyr via des API dédiées, permettant un réveil « à la demande ».
- **Gestion centralisée par le système (*System-Managed Device Power Management*)** : Dans ce mode, le noyau coordonne la mise en veille des périphériques en synchronisation avec celle du processeur (CPU). Lorsqu'une transition vers un état de basse consommation global est décidée, le système se charge de suspendre automatiquement tous les périphériques inactifs. Cette suspension s'effectue strictement dans l'ordre inverse de leur initialisation, afin de préserver les dépendances matérielles et de garantir la stabilité du système.

5.3.2 Connectivité et protocoles

Pour répondre aux exigences de l'Internet des Objets, Zephyr embarque nativement une pile réseau complète, modulaire et hautement optimisée, appelée le *Net Subsystem*. Cette pile est conçue pour s'adapter aux architectures à faibles ressources tout en offrant des fonctionnalités dignes des systèmes d'exploitation majeurs.

L'architecture réseau de Zephyr fonctionne avec plusieurs couches clé :

- **L'interface applicative (API)** : Zephyr expose une interface de programmation basée sur le standard POSIX (*BSD Sockets*). Ce choix architectural est crucial, car il permet aux développeurs de créer des applications réseau de haut niveau (utilisant CoAP, MQTT ou HTTP) de manière totalement agnostique vis-à-vis du matériel et des protocoles sous-jacents.
- **Le cœur réseau (Couches 3 et 4)** : Le système gère nativement le routage IPv4 et IPv6, ainsi que les protocoles de transport TCP et UDP.
- **La couche liaison de données (Couche 2)** : Le *Net Subsystem* supporte une grande variété de technologies physiques et de protocoles de liaison, tels que l'Ethernet, le Wi-Fi, le Bluetooth Low Energy (BLE) et la norme radio IEEE 802.15.4.

Bien que Zephyr intègre nativement la norme IEEE 802.15.4 et supporte des protocoles maillés comme Thread (via l'intégration d'OpenThread), il souffre actuellement d'un manque important pour les déploiements urbains à grande échelle : il n'existe à ce jour aucune implémentation open source native du standard **Wi-SUN** dans son écosystème.

C'est précisément cette lacune que le portage de la Nanostack vient combler.

5.4 Nanostack

La Nanostack est la pile réseau développée originellement pour Mbed OS. Mbed OS arrivant en fin de vie, la migration de cette pile logicielle vers un nouvel OS est devenue indispensable.

Pour mener à bien ce portage, il est important d'appréhender l'architecture interne de la Nanostack. Celle-ci repose sur une séparation stricte des responsabilités, divisée en deux couches principales, animées par un moteur d'événements :

- La **SAL** (*Software Abstraction Layer*) : La partie purement logicielle et applicative.
- La **HAL** (*Hardware Abstraction Layer*) : L'interface avec les composants matériels.
- Le **Nanostack Event Loop** : Le cœur asynchrone qui cadence l'ensemble des opérations.

5.4.1 SAL (Software Abstraction Layer)

La SAL gère toute la partie purement logicielle de la Nanostack, totalement indépendante du matériel (qui est délégué à la couche matérielle, la HAL). Son rôle principal est de masquer la complexité du réseau en offrant une interface de programmation standardisée (API Socket) à l'application.

C'est dans cette couche que s'exécutent les machines d'état des différents protocoles (MAC, 6LoWPAN, routage RPL, IPv6, TCP/UDP). Pour fonctionner efficacement sur des microcontrôleurs limités, la SAL s'appuie sur trois piliers :

- **Le Nanostack Event Loop** : Un ordonnanceur d'événements coopératif qui gère le trafic réseau de façon asynchrone pour économiser la RAM, sans nécessiter de multiples *threads*.
- **Une gestion interne du temps et de la mémoire** (`ns_timer` et `ns_dyn_mem`) : Un système de minuteurs virtuels pour les *timeouts* réseau et un allocateur de *buffers* optimisé pour prévenir la fragmentation de la mémoire lors d'un trafic intense.

Enfin, la SAL intègre nativement les mécanismes de sécurité (via Mbed TLS) pour chiffrer les communications avant leur transmission. Il est important de noter que bien que Mbed OS ait atteint sa fin de vie, le projet Mbed TLS demeure activement maintenu par la

communauté TrustedFirmware [10]. Son intégration reste donc pertinente et pérenne pour ce projet, nous dispensant de chercher une solution cryptographique alternative.

5.4.2 HAL (Hardware Abstraction Layer)

La HAL fait le pont entre le système d'exploitation hôte (Zephyr) et la Nanostack. Son rôle est de relier la logique logicielle de la pile réseau aux composants physiques. C'est à ce niveau que se fait l'interface avec les pilotes matériels (*drivers*), indispensables pour communiquer via Ethernet ou via la norme IEEE 802.15.4.

Pour réussir le portage vers Zephyr, l'implémentation de la HAL doit couvrir plusieurs domaines critiques :

- **L'allocation de la mémoire** : Bien que la Nanostack possède son propre gestionnaire de mémoire interne, elle doit interagir avec le noyau du RTOS hôte lors de son initialisation pour réserver son bloc de mémoire principal (via l'allocation dynamique de Zephyr ou l'assignation d'un bloc statique).
- **La gestion de la concurrence (Multithreading)** : Mbed OS et Zephyr étant tous deux des RTOS, la HAL doit traduire les primitives de synchronisation. Il est impératif d'implémenter les mutex, les sémaphores et les sections critiques pour protéger l'état de la Nanostack contre les accès concurrents provenant de différents *threads*.
- **Les horloges matérielles et interruptions** : La HAL doit faire le lien entre les minuteurs (*timers*) matériels gérés par Zephyr et les événements temporels de la Nanostack, tout en gérant les interruptions matérielles (par exemple, signaler à la pile logicielle qu'un paquet radio vient d'être physiquement reçu).
- **Génération de nombres aléatoires** : La **Nanostack** requiert une source d'entropie pour garantir la non-prédictibilité des identifiants IPv6, la sécurité des clés cryptographiques et la gestion des mécanismes de *backoff* lors des transmissions radio. La HAL expose ici le générateur de nombres aléatoires matériel de **Zephyr** afin d'offrir une source de données sécurisée et robuste aux algorithmes de la pile.

5.4.3 Le moteur d'événements (*Nanostack Event Loop*)

Au cœur de l'architecture de la Nanostack se trouve un composant central : le *Nanostack Event Loop*. Plutôt que de créer un fil d'exécution (*thread*) pour chaque connexion ou tâche réseau, ce qui serait bien trop coûteux en mémoire pour un microcontrôleur, la pile utilise un ordonnanceur d'événements coopératif.

Toutes les actions du réseau (réception d'un paquet, expiration d'un délai, demande de l'application) sont converties en événements et placées dans une file d'attente globale (*Event Queue*). L'Event Loop dépile ensuite ces événements un par un de manière asynchrone et

déclenche les fonctions de rappel (*callbacks*) associées. Ce fonctionnement garantit une utilisation minimale et déterministe de la RAM.

5.4.4 Dépendance au système d'exploitation hôte (CMSIS-RTOS)

Bien que la Nanostack soit agnostique vis-à-vis du matériel, elle dépend intrinsèquement des services du système d'exploitation sur lequel elle tourne pour ses opérations fondamentales. Dans son environnement d'origine (Mbed OS), cette dépendance est gérée via l'API standardisée **CMSIS-RTOS**.

La pile réseau fait régulièrement appel à cette couche de compatibilité pour trois fonctions critiques :

- **La synchronisation** : Utilisation de verrous (*mutexes*) pour protéger ses structures de données internes lorsque des requêtes proviennent de différents *threads* applicatifs.
- **L'allocation mémoire** : Réservation de la mémoire au démarrage pour son allocateur interne (`ns_dyn_mem`).
- **La gestion du temps** : Interfaçage avec les horloges du système pour la gestion des *timeouts* et le cadencement de l'Event Loop.

Enjeu pour le portage : C'est sur ce point précis que se situe le cœur de la migration. Les appels à CMSIS-RTOS étant profondément ancrés dans la couche d'adaptation de la Nanostack, l'objectif du projet sera de substituer ces appels par les primitives natives de Zephyr (comme les API `k_mutex` ou `k_timer`), assurant ainsi une intégration fluide sans dénaturer le code source des protocoles.

5.4.5 Réception des paquets sous Mbed OS

Pour comprendre les enjeux du portage, il est essentiel d'analyser le flux de données actuel lors de la réception d'un paquet sous Mbed OS. Mbed OS étant majoritairement orienté objet (C++), tandis que la Nanostack est écrite en C, le passage du monde matériel au monde réseau nécessite une interface de traduction.

Le flux de réception d'une trame radio (ex: IEEE 802.15.4) se déroule en quatre étapes :

1. **L'interruption matérielle (ISR)** : Lorsque la puce radio reçoit un paquet valide, elle lève une interruption matérielle. Le noyau Mbed OS intercepte cette IRQ et exécute la routine d'interruption du pilote radio.
2. **L'interface matérielle (NanostackRfPhy)** : Mbed OS utilise une classe abstraite en C++ nommée `NanostackRfPhy`. Le pilote radio spécifique à la carte (par exemple, un driver Atmel ou STM32) hérite de cette classe. C'est ce pilote qui va lire les données brutes sur le bus matériel (SPI/UART).

3. **Le pont C/C++ et la fonction de rappel (*Callback*)** : Lors de l'initialisation du système, la classe `NanostackRfPhy` enregistre le périphérique radio auprès de la couche C de la Nanostack via l'API `arm_net_phy_register()`. Cette fonction fournit au pilote un pointeur vers une fonction de réception interne à la pile (souvent `phy_rx_cb`). Le pilote Mbed OS appelle cette fonction C en lui passant les données du paquet.
4. **L'Event Loop dans un *Thread* Mbed OS** : La fonction de rappel ne traite pas la trame directement. Elle alloue un tampon avec `ns_dyn_mem`, y copie la charge utile, et poste un événement réseau dans le *Nanostack Event Loop*. Sous Mbed OS, cet *Event Loop* s'exécute en continu à l'intérieur d'un *thread* CMSIS-RTOS dédié (souvent nommé `ns_thread`). Le RTOS réveille ce *thread*, qui dépile l'événement et fait remonter le paquet de manière asynchrone vers les couches MAC, 6LoWPAN et IPv6.

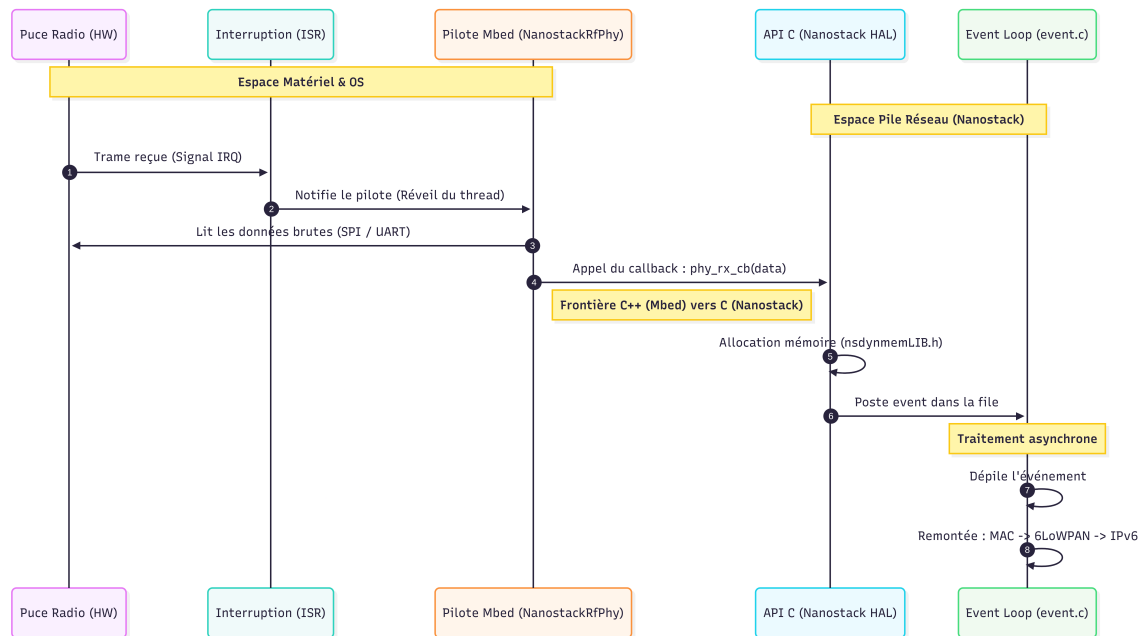


Fig. 2. – Transaction driver -> Nanostack

L'architecture actuelle démontre que la Nanostack n'a aucune conscience de la classe C++ `NanostackRfPhy` ni du fonctionnement interne de l'ISR de Mbed OS. Elle s'attend uniquement à ce qu'un code externe (la HAL) appelle sa fonction de rappel C (`phy_rx_cb`) et lui fournisse des événements. Dans Zephyr, il suffira de reproduire ce comportement depuis l'API `ieee802154 native`.

5.5 Conclusion de l'état de l'art

L'analyse de ces différentes technologies met en évidence la pertinence du portage de la Nanostack vers Zephyr. D'un côté, le standard Wi-SUN répond parfaitement aux exigences des réseaux urbains modernes. De l'autre, Zephyr offre une base temps réel robuste, modulaire et taillée pour l'efficacité énergétique, mais manque encore d'une implémentation Wi-SUN native. L'architecture interne de la Nanostack, séparant habilement la logique réseau (SAL) des interactions matérielles (HAL), rend cette intégration réalisable. Le défi technique ne réside donc pas dans la réécriture des protocoles, mais bien dans la conception des interfaces (adaptateurs) entre la HAL de la Nanostack et les API natives de Zephyr.

6 Méthodologie et architecture du portage

6.1 Intégration logicielle : Bibliothèque vs Module Zephyr

Lors du portage d'une base de code aussi volumineuse que la Nanostack, la méthode d'intégration au système de compilation est un choix d'architecture critique. Zephyr propose deux approches principales pour inclure du code tiers : l'intégration en tant que bibliothèque statique au sein de l'application, ou la création d'un Module Zephyr externe [11] .

6.1.1 L'approche Bibliothèque classique (*In-Tree*)

Cette méthode consisterait à copier l'intégralité des fichiers sources de la Nanostack directement dans le dossier de l'application finale, et à configurer le fichier CMakeLists.txt de l'application pour les compiler.

- **Avantage** : Mise en place initiale très rapide.
- **Inconvénient** : Cette approche lie fortement la pile réseau à l'application métier. Elle rend la mise à jour de la Nanostack difficile, pollue l'historique de version (Git) du projet principal et rend la réutilisation de la pile par d'autres projets quasiment impossible.

6.1.2 L'approche Module Zephyr (*Out-of-Tree*)

Un Module Zephyr est un dépôt de code (par exemple hébergé sur GitHub) totalement indépendant de l'arbre source de Zephyr et de l'application finale. Il est automatiquement téléchargé et lié lors de la configuration du projet grâce à l'outil de gestion de paquets west (via le fichier west.yml).

- **Séparation des responsabilités** : Le code de la Nanostack reste isolé. L'application métier se contente de l'appeler via les API publiques.
- **Intégration native (Kconfig et CMake)** : Un module permet de définir ses propres menus de configuration (Kconfig). Ainsi, les paramètres de la Nanostack (activation du Wi-

SUN, taille des *buffers*) apparaîtront directement dans les menus de configuration de Zephyr (via `menuconfig`), exactement comme les sous-systèmes natifs.

- **Portabilité et partage** : En plaçant la Nanostack dans son propre dépôt Git structuré comme un module, elle devient facilement distribuable à l'ensemble de la communauté open source.

6.1.3 Comparaison des approches

Afin de justifier le choix de l'architecture logicielle, le tableau suivant dresse le bilan des avantages et inconvénients des deux méthodes d'intégration au sein de l'écosystème Zephyr :

Approche	Avantages	Inconvénients
Bibliothèque classique (<i>In-Tree</i>)	• Mise en place initiale rapide et intuitive. • Ne requiert pas d'outils externes (uniquement CMake). • Utile pour un prototypage très basique et isolé.	• Forte dépendance logicielle (couplage fort) entre l'application et la pile réseau. • Mise à jour future de la Nanostack très complexe. • Pollution de l'historique Git du projet principal. • Aucune intégration native avec les menus de configuration Kconfig. • Réutilisation par d'autres projets plus ardue.
Module Zephyr (<i>Out-of-Tree</i>)	• Séparation stricte des responsabilités. • Intégration native aux menus Kconfig de Zephyr. • Gestion des dépendances automatisée via l'outil <code>west</code>. • Versionnement Git totalement indépendant. • Code standardisé et facilement distribuable.	• Courbe d'apprentissage initiale légèrement plus rude (nécessite de maîtriser le format <code>module.yml</code>). • Configuration initiale des fichiers de liaison plus verbeuse.

Tableau 2. – Comparaison des méthodes d'intégration logicielle sous Zephyr

6.1.4 Décision architecturale et Manifeste `west`

Pour ce projet de Bachelor, l'approche par Module **Zephyr** est retenue. Elle est la seule à garantir une architecture logicielle pérenne. Le choix de cette architecture en module externe (*Out-of-Tree*) implique l'utilisation avancée du système de manifeste.

L'application finale contiendra un fichier `west.yml`. Lors de l'initialisation de l'environnement (via la commande `west update`), l'outil se chargera de cloner le dépôt de la Nanostack, de l'insérer dans l'espace de travail (*workspace*), et de parser son fichier `zephyr/module.yml` pour lier ses menus et scripts de compilation au système principal.

6.2 Implémentation de la structure du module

Pour être reconnu par le système de compilation de Zephyr, le module externe doit respecter une topologie stricte et fournir trois fichiers fondamentaux : `module.yml` (description), `CMakeLists.txt` (compilation) et `Kconfig` (configuration).

La topologie classique du module développé pour ce portage est la suivante :

```
zephyr-nanostack/
├── zephyr/
│   └── module.yml    <-- Déclaration du module pour west
├── include/         <-- En-têtes publiques (API de la Nanostack)
├── src/              <-- Code source C (event_loop, adaptateurs...)
├── CMakeLists.txt    <-- Règles de compilation Zephyr
└── Kconfig           <-- Options d'activation pour le menuconfig
```

Liste 1. – Arborescence type du module Zephyr pour la Nanostack

Côté application, il suffira de modifier le fichier `west.yml` du projet cible pour y indiquer cette nouvelle dépendance et son URL Git :

```
1 manifest:
2 projects:a
3   - name: zephyr-nanostack
4     url: [https://github.com/Travail-de-Bachelor-Delay-Benoit/zephyr-nanostack.git](https://github.com/Travail-de-Bachelor-Delay-Benoit/zephyr-nanostack.git)
5     revision: main
```

Listing 2. – Déclaration du module dans le `west.yml` de l'application

Une fois le module téléchargé, l'activation de la pile réseau se fera via le système de configuration standard. Le fichier Kconfig du module exposera les options nécessaires :

```
1 menuconfig NANOSTACK
2     bool "Support de la pile réseau Nanostack (Wi-SUN)"
3     default n
4     help
5         Active la compilation et l'intégration de la Nanostack.
```

Listing 3. – Fichier Kconfig à la racine du module

Enfin, le développeur n'aura plus qu'à ajouter `CONFIG_NANOSTACK=y` dans le fichier `prj.conf` de son application pour que la pile soit automatiquement incluse à la compilation.

6.3 Adaptation du moteur d'événements (*Event Loop*)

L'un des défis majeurs du portage consiste à substituer les primitives de Mbed OS (CMSIS-RTOS) par celles de Zephyr. Le cœur de la Nanostack reposant sur un moteur d'événements (*Event Loop*), ce dernier doit s'exécuter dans son propre fil d'exécution (*thread*).

Sous Zephyr, ce fil d'exécution sera instancié à l'aide de l'API native `K_THREAD_DEFINE` ou `k_thread_create`. Il est crucial que l'implémentation de ce *thread* prenne rigoureusement en compte la politique de gestion de l'énergie. En effet, la basse consommation étant l'un des piliers du standard Wi-SUN, la boucle d'événements ne doit en aucun cas monopoliser le processeur. Lorsqu'aucun événement réseau n'est présent dans la file d'attente, le *thread* de la Nanostack devra explicitement relâcher la main (via `k_sleep` ou en bloquant sur une primitive de synchronisation) afin de permettre à Zephyr de basculer le système en veille profonde (*Deep Sleep*).

6.4 Intégration cryptographique (Mbed TLS)

Le standard Wi-SUN impose des exigences de sécurité strictes, nécessitant l'authentification des nœuds (EAP-TLS) et le chiffrement des trames. La Nanostack s'appuie historiquement sur le projet **Mbed TLS** pour sa partie cryptographique.

Cette dépendance s'intègre parfaitement à la stratégie de portage puisque Zephyr utilise également Mbed TLS comme bibliothèque cryptographique officielle par défaut. Le portage ne nécessitera donc pas de réécrire les algorithmes de sécurité. Il consistera principalement à s'assurer que le système de compilation du module (via CMake et Kconfig) force l'activation de Mbed TLS (`CONFIG_MBEDTLS=y`).

De plus, une attention particulière devra être portée sur l'entropie matérielle. Pour générer des clés cryptographiques robustes, la pile devra être correctement interfacée avec le générateur de nombres aléatoires matériel exposé par les API de Zephyr.

6.5 Refonte des règles de compilation (CMakeLists)

La dernière étape stratégique concerne l'adaptation du processus de compilation. Mbed OS utilisait son propre système d'assemblage, qui doit être entièrement remplacé par des directives CMake compatibles avec l'écosystème Zephyr.

Le fichier CMakeLists.txt situé à la racine du module exploitera les macros spécifiques de l'OS (comme `zephyr_library()` et `zephyr_library_sources()`). Ces directives permettront d'inclure conditionnellement les fichiers sources en langage C de la Nanostack uniquement si l'option `CONFIG_NANOSTACK` est sélectionnée par l'utilisateur, garantissant ainsi que le code mort n'encombre pas la mémoire flash des applications ne nécessitant pas le Wi-SUN.

De plus, une attention particulière devra être accordée à la compilation du cœur de la Nanostack (les parties totalement agnostiques et indépendantes du système d'exploitation). Il sera également impératif de refléter avec précision la nouvelle arborescence des fichiers dans les directives CMake, afin de garantir la bonne résolution des chemins d'inclusion (*headers*) et de préserver l'intégrité de l'architecture logicielle.

7 Implémentation

7.1 Compilation de la Nanostack

Adapter la **Nanostack**, initialement faite pour **Mbed OS**, à **Zephyr** a été le premier gros chantier. Bien que les deux utilisent **CMake**, ils fonctionnent différemment : **Mbed** est très décentralisé avec ses fichiers de build dans chaque dossier, alors que **Zephyr** centralise tout via son noyau.

Afin de préserver l'intégrité de la base de code d'origine et d'éviter une maintenance complexe, le choix a été fait de l'encapsuler dans un module dédié, Zephyr-Nanostack, via un fichier maître orchestrant la compilation.

7.1.1 Intégration à l'interface de configuration Kconfig

En plus de **CMake**, il fallait que la stack s'intègre dans le système de configuration de **Zephyr** (**Kconfig** et **Kbuild**). L'idée est de pouvoir activer ou désactiver la stack et ses composants directement via `menuconfig` ou les fichiers `prj.conf`.

```
menuconfig NANOSTACK
  bool "Intégration de la Nanostack"
  select MBEDTLS
  select MBEDTLS_ENTROPY_C
  default n

if NANOSTACK
  config NANOSTACK_BRIDGE_RADIO
    bool "Utiliser le pont radio (AF_PACKET)"
    default n

    config NANOSTACK_BRIDGE_RADIO_DIRECT
      bool "Utiliser le pont radio direct"
      default n
endif
```

Listing 4. – Structure du fichier Kconfig

L'avantage est notable : grâce aux directives `select`, **Zephyr** gère les dépendances de manière autonome. Chaque option crée une macro C (`CONFIG_NANOSTACK_BRIDGE_RADIO`) utilisable directement dans le code avec des `#ifdef`.

7.1.2 Le fichier `CMakeLists.txt` maître

7.1.2.1 Gestion conditionnelle de la compilation

Pour que le build reste léger, l'inclusion de la stack est conditionnée. Si l'option n'est pas activée, le compilateur n'accède pas au dossier.

```
if(CONFIG_NANOSTACK)
  add_subdirectory(nanostack)
endif()
```

Listing 5. – Compilation conditionnelle

7.1.2.2 Gestion des dépendances Mbed

Intégrer les composants de **Mbed** (comme `mbed-trace` ou `coap-service`) a été technique : il fallait que **Zephyr** puisse compiler ces modules sans modifier leurs fichiers sources. Pour chaque composant, il a été nécessaire d'extraire les propriétés d'interface, de déclarer les répertoires pour les en-têtes, et lier le tout à la bibliothèque cible.

```
zephyr_library_include_directories(  
    include  
    src/nanostack-libservice  
    src/mbed-trace/include  
    zephyr_port/include  
    $<TARGET_PROPERTY:mbed-core,INTERFACE_INCLUDE_DIRECTORIES>  
)  
  
add_subdirectory(src/mbed-trace)  
  
zephyr_library_sources(  
    $<TARGET_PROPERTY:mbed-core,INTERFACE_SOURCES>  
    src/sample.c  
)
```

Listing 6. – Liaison dynamique des dépendances

7.2 Architecture du portage et gestion des conflits

Pour garantir une structure propre, l'ensemble des mécanismes d'adaptation a été isolé au sein d'un répertoire dédié, `zephyr_port`. Cette séparation physique est un choix architectural clé pour assurer l'indépendance du code source original et faciliter toute mise à jour ultérieure. Un développeur tiers peut ainsi immédiatement distinguer la logique réseau native de l'adaptation système, qui concentre toute la complexité liée aux API du RTOS.

7.2.1 Gestion des conflits de nommage : l'approche par « Glue Code »

Lors de l'intégration, un conflit de nommage critique s'est présenté : le mot-clé `device`, largement utilisé dans la **Nanostack**, entrainait en collision avec le fichier d'en-tête `zephyr/device.h`. Pour éviter de renommer des milliers d'occurrences, ce qui aurait rendu toute mise à jour difficilement gérable, une méthode de redirection en deux temps a été mise en place.

D'abord, un fichier `glue.h` a été créé pour faire office de « couche de réécriture » :

```
#include <zephyr/kernel.h>  
#include <zephyr/device.h>  
  
// On renomme device pour éviter la collision  
#define device ns_device  
  
#include "zephyr_port.h"
```

Listing 7. – Gestion du conflit device via le fichier glue.h

Ensuite, pour appliquer cette règle de manière globale sans altérer les fichiers sources `.c`, le système de build **CMake** a été configuré pour forcer l'inclusion automatique de ce fichier d'en-tête via l'option `-include`. Cette approche est totalement transparente et préserve l'intégrité de la pile originale.

```
target_compile_options(zephyr_library_nanostack PRIVATE
    -include ${PORT_DIR}/include/glue.h
    -fcommon
    -Wno-error=inline
)
```

Listing 8. – Forcer l'inclusion de `glue.h` via CMake

7.3 Abstraction des services système (OS Abstraction Layer)

Comme prévu dans l'état de l'art, la **Nanostack** nécessite une couche d'adaptation pour ses appels système. C'est l'OS Abstraction Layer : un pont entre les besoins de la stack et les services de **Zephyr**.

7.3.1 Gestion de la concurrence

Pour éviter les problèmes d'accès concurrents (*race conditions*), un système de sections critiques réentrantes a été implémenté[12].

```
static unsigned int irq_key;
static volatile int nesting_level = 0;

void platform_enter_critical() {
    unsigned int key = irq_lock();
    if (nesting_level == 0) irq_key = key;
    nesting_level++;
}

void platform_exit_critical() {
    if (nesting_level > 0 && --nesting_level == 0) {
        irq_unlock(irq_key);
    }
}

K_SEM_DEFINE(ns_event_sem, 0, 1);
```

Listing 9. – Gestion des sections critiques

Un compteur (`nesting_level`) est utilisé pour gérer l'imbrication et un sémaphore est mis en œuvre pour réveiller la pile sans bloquer le reste du système.

7.3.2 Gestion temporelle (Timers)

La stack travaille avec des tranches temporelles (« slots ») de 50 μ s. Ce comportement a été mappé sur les timers de **Zephyr**[13] en convertissant les slots en microsecondes, avec un arrondi supérieur pour conserver la précision.

```
void platform_timer_start(uint16_t slots) {
    k_timer_start(&nanostack_timer, K_USEC(slots * 50), K_NO_WAIT);
}

uint16_t platform_timer_get_remaining_slots(void) {
    k_ticks_t remaining = k_timer_remaining_ticks(&nanostack_timer);
    uint32_t us = k_ticks_to_us_floor32(remaining);
    return (uint16_t)((us + 49) / 50);
}
```

Listing 10. – Mapping des timers

7.3.3 Génération de nombres aléatoires

Enfin, pour l'aléa, la stack d'origine utilisait une gestion manuelle de la graine. Sous **Zephyr**[14], le noyau gère cela de manière plus sécurisée via son générateur de nombres aléatoires matériel. La sécurité a donc été déléguée au système d'exploitation en laissant vides les fonctions de « seed » manuelles.

```
static inline void randLIB_seed_random(void) {}
static inline void randLIB_add_seed(uint64_t seed) {}

static inline uint16_t randLIB_get_16bit(void) {
    return sys_rand16_get();
}
```

Listing 11. – Abstraction de l'aléa

7.3.4 Intégration de la cryptographie (Mbed TLS)

La pile réseau Nanostack requiert des primitives de chiffrement symétrique AES-CCM-128 pour sécuriser les trames de données de niveau 2. Sous Mbed OS, ces opérations s'appuient sur la bibliothèque **Mbed TLS**. Afin de réaliser ce portage sous Zephyr, les appels cryptographiques propriétaires de la Nanostack (fonctions `arm_aes_start`, `arm_aes_encrypt` et

`arm_aes_finish`) ont été mappés vers l'implémentation native de **Mbed TLS** fournie par le noyau Zephyr.

Cette intégration s'organise autour des points techniques suivants :

7.3.4.1 Déclarations Kconfig de Zephyr

L'intégration de Mbed TLS s'effectue de manière transparente en configurant les options requises dans le fichier Kconfig de notre module. Grâce aux sélections automatiques (Listing 12), Zephyr se charge de compiler la bibliothèque et de lier le générateur de nombres aléatoires matériel de la puce cible.

```
menuconfig NANOSTACK
    bool "Intégration de la Nanostack"
    select MBEDTLS
    select MBEDTLS_ENTROPY_C
```

Listing 12. – Sélection de Mbed TLS via Kconfig

7.3.4.2 Allocation statique des contextes AES (`zephyr_port.c`)

La Nanostack requiert l'allocation d'une structure `mbedtls_aes_context` à chaque ouverture de session de chiffrement. Pour éviter l'utilisation de `malloc` qui fragmenterait la mémoire vive (RAM) du microcontrôleur lors du traitement intensif de paquets, un pool statique de contextes réentrants a été implémenté (Listing 13) :

- Un tableau statique `context_list` d'une taille minimale est déclaré en mémoire.
- La fonction d'allocation `mbed_tls_context_get` parcourt ce pool et réserve un contexte libre sous la protection d'une section critique réentrante (`platform_enter_critical`), évitant ainsi toute corruption par un thread concurrent.
- `arm_aes_start` initialise le contexte avec la clé secrète de 128 bits.
- Une fois le chiffrement terminé par `arm_aes_encrypt`, la fonction `arm_aes_finish` libère le contexte Mbed TLS et remet le drapeau `reserved` à faux pour les futures trames.

```

struct arm_aes_context {
    mbedtls_aes_context ctx;
    bool reserved;
};
static struct arm_aes_context context_list[ARM_AES_MBEDTLS_CONTEXT_MIN];

static struct arm_aes_context *mbed_tls_context_get(void) {
    platform_enter_critical();
    for (int i = 0; i < ARM_AES_MBEDTLS_CONTEXT_MIN; i++) {
        if (!context_list[i].reserved) {
            context_list[i].reserved = true;
            platform_exit_critical();
            return &context_list[i];
        }
    }
    platform_exit_critical();
    return NULL;
}

void *arm_aes_start(const uint8_t *key) {
    struct arm_aes_context *context = mbed_tls_context_get();
    if (context) {
        mbedtls_aes_init(&context->ctx);
        if (0 != mbedtls_aes_setkey_enc(&context->ctx, key, 128)) {
            mbedtls_aes_free(&context->ctx);
            platform_enter_critical();
            context->reserved = false;
            platform_exit_critical();
            return NULL;
        }
    }
    return (void *)context;
}

```

Listing 13. – Gestion statique et réentrant des contextes de chiffrement AES

7.3.4.3 Implémentation des stubs POSIX

Certains fichiers de la bibliothèque **Mbed TLS** compilés pour des architectures génériques font référence à des appels système d'accès aux fichiers (POSIX). N'ayant pas besoin de système de fichiers pour la gestion en mémoire de la cryptographie réseau, des fonctions de substitution (**stubs**) minimales (open, close, read, write, lseek, unlink) retournant simplement -1 ont été implémentées. Cela permet de satisfaire le lieur (**linker**) sans alourdir le binaire final.

7.4 Couche d'adaptation L2 (Data Link Layer)

La **Nanostack** doit pouvoir s'appuyer sur les pilotes matériels natifs de **Zephyr** pour émettre et recevoir des données. L'objectif ici était de créer un pont (**bridge**) entre l'API de réception de la **Nanostack** (la couche MAC) et les **drivers** L2 de **Zephyr** (Ethernet et IEEE 802.15.4).

7.4.1 Mécanisme de pontage réseau

Pour chaque interface, un mécanisme de transfert de paquets bidirectionnel a été mis en place. Concrètement, le pont doit traiter deux flux :

- **Flux descendant (Tx) :** La **Nanostack** envoie des trames formatées. Celles-ci doivent être traduites pour être acceptées par l'interface réseau de **Zephyr**.
- **Flux montant (Rx) :** À chaque réception d'une trame par le driver de **Zephyr**, le paquet est encapsulé et redirigé vers la pile réseau de la **Nanostack** pour traitement.

L'utilisation de deux types de ponts répond à des besoins spécifiques :

- **Pont Ethernet(AF_PACKET) :** Utilisé pour acheminer les trames Ethernet, ce pont exploite l'interface AF_PACKET de Zephyr pour capturer et injecter des trames de couche 2 (Data Link Layer) directement au sein de la pile réseau.
- **Pont radio (AF_PACKET) :** Utilisé pour les communications sans fil, où la gestion des adresses MAC et la fragmentation sont critiques. Ici, le pont permet de faire transiter des trames brutes (raw frames) directement vers le driver IEEE 802.15.4.
- **Pont radio direct :** Une version simplifiée du pontage, utilisée pour minimiser la latence en court-circuitant certaines vérifications logicielles.

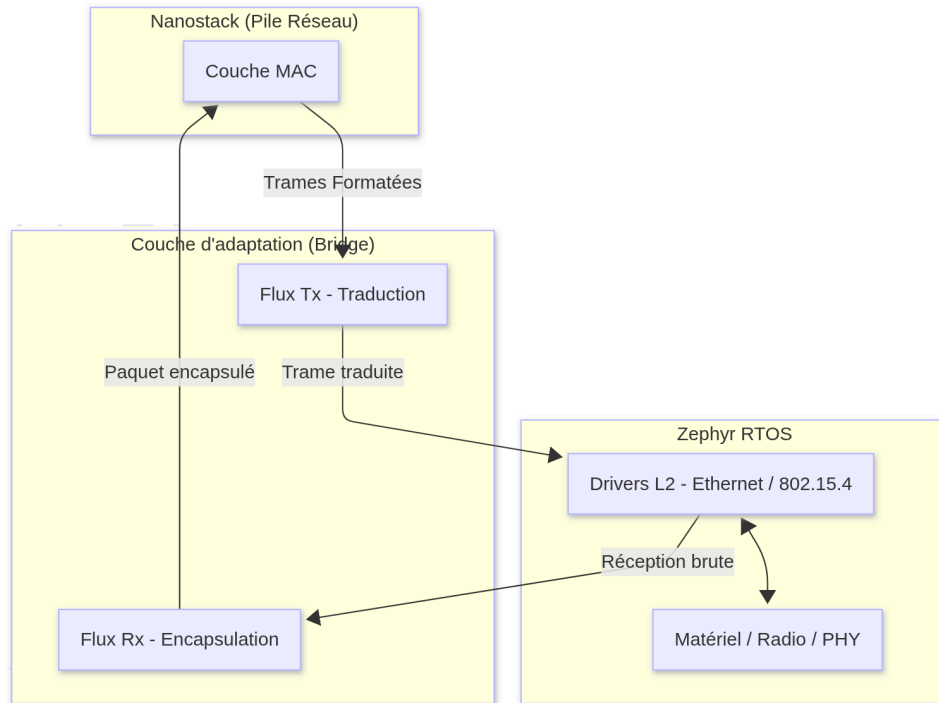


Fig. 3. – Architecture du mécanisme de pontage réseau

Cette adaptation est transparente pour les deux systèmes. La **Nanostack** croit toujours piloter une interface radio ou Ethernet, alors qu'en réalité, elle s'appuie sur la pile réseau de **Zephyr** pour accéder aux ressources matérielles. Cette architecture de « bridge » permet de conserver toute la logique de routage de la **Nanostack** tout en bénéficiant de la fiabilité des drivers **Zephyr** et ainsi éviter la réécriture de tous les drivers.

7.4.2 Architecture et configuration des ponts réseaux

Le module **zephyr-nanostack** intègre directement la définition et l'implémentation des ponts nécessaires à la communication. Ces composants sont exposés via des fichiers d'en-tête dédiés, garantissant une architecture propre et modulaire.

Pour optimiser l'empreinte mémoire et ne compiler que les composants strictement nécessaires, le choix du pont s'effectue au moment de la configuration via le système **Kconfig**. Ce mécanisme permet de définir le comportement réseau du système dans le fichier `prj.conf` ou via l'interface `menuconfig`.

Le pont Ethernet est activé par défaut si aucune autre option n'est spécifiée, assurant une compatibilité immédiate avec les réseaux filaires. Le pont radio utilisant `AF_PACKET` est activé

via l'option `NANOSTACK_BRIDGE_RADIO`, permettant de faire transiter les trames vers le driver IEEE 802.15.4 de **Zephyr**. Enfin, le pont radio direct est accessible via `NANOSTACK_BRIDGE_RADIO_DIRECT` ; cette variante court-circuite certaines étapes logicielles pour minimiser la latence.

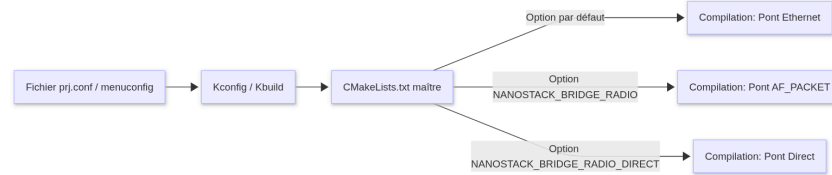


Fig. 4. – Flux de configuration des ponts réseaux

Cette structure offre une grande souplesse. Le développeur peut basculer d'une technologie réseau à une autre sans modifier le code source, mais simplement en ajustant les paramètres de build. Le système de **Kconfig** garantit ainsi qu'aucune logique inutile n'est intégrée dans l'image binaire finale, respectant les contraintes de ressources propres aux systèmes embarqués.

7.4.3 Intégration de la liaison Ethernet (`bridge_l2.c`)

Afin d'assurer la liaison entre le pilote matériel (couche L1), la couche de liaison de Zephyr (couche L2) et la pile Nanostack (qui implémente ses propres couches supérieures L2/L3), il est nécessaire de concevoir une couche d'adaptation. L'objectif est de permettre à la Nanostack de recevoir et d'émettre des trames physiques via le pilote matériel. Dans un premier temps, les recherches se sont orientées vers l'utilisation de l'API de gestion de la couche de liaison (L2 Layer Management)[15] de Zephyr. Cependant, il est rapidement apparu que les pilotes de périphériques réseau sont liés statiquement et de manière immuable à la couche L2 native de Zephyr (`ETHERNET_L2`). Ce couplage rend impossible tout court-circuitage dynamique à l'exécution, comme le met en évidence la structure de données `net_if_dev` de Zephyr présentée dans le Listing 14.

```

struct net_if_dev {
    /** Périphérique matériel associé à l'interface réseau */
    const struct device *dev;
    /** Pointeur strictement constant vers la couche L2 de l'interface */
    const struct net_l2 * const l2;
    /** Pointeur vers les données privées de la couche L2 */
    void *l2_data;
    /* ... Reste de la structure ... */
};

```

Listing 14. – Structure `net_if_dev` de Zephyr

Dans cette structure, la déclaration `const struct net_l2 * const l2;` applique une double contrainte de lecture seule : le pointeur `l2` ainsi que la structure d'API L2 pointée sont définis comme constants. Ces informations sont figées à la compilation, stockées en mémoire Flash (ROM) et ne peuvent donc pas être réassignées en cours d'exécution. Cette assignation statique provient des macros d'initialisation réseau appelées par le pilote lors de sa compilation, présentées dans le Listing 15 :

```

#define Z_ETH_NET_DEVICE_INIT_INSTANCE(node_id, dev_id, name, instance, \
    init_fn, pm, data, config, prio, \
    api, mtu) \
    Z_NET_DEVICE_INIT_INSTANCE(node_id, dev_id, name, instance, \
    init_fn, pm, data, config, prio, \
    api, ETHERNET_L2, \
    NET_L2_GET_CTX_TYPE(ETHERNET_L2), mtu)

```

Listing 15. – Macro d'initialisation d'une interface Ethernet dans Zephyr

Comme l'illustre ce Listing, le symbole `ETHERNET_L2` est passé directement en paramètre de macro lors de l'enregistrement de l'interface. Par conséquent, utiliser une couche L2 alternative (comme celle requise par la Nanostack) directement sur l'interface réseau exigerait de modifier le code source de tous les pilotes de périphériques pour y injecter une autre macro d'initialisation. Pour éviter cette réécriture invasive et préserver l'agnosticisme matériel des pilotes, l'implémentation d'un pont logiciel s'est avérée indispensable. La trame brute est capturée à l'aide d'un socket brut (`AF_PACKET`) et transmise à la Nanostack, contournant ainsi la couche L2 de Zephyr sans altérer le pilote, comme décrit sur le schéma d'architecture ci-dessous (Fig. 5).

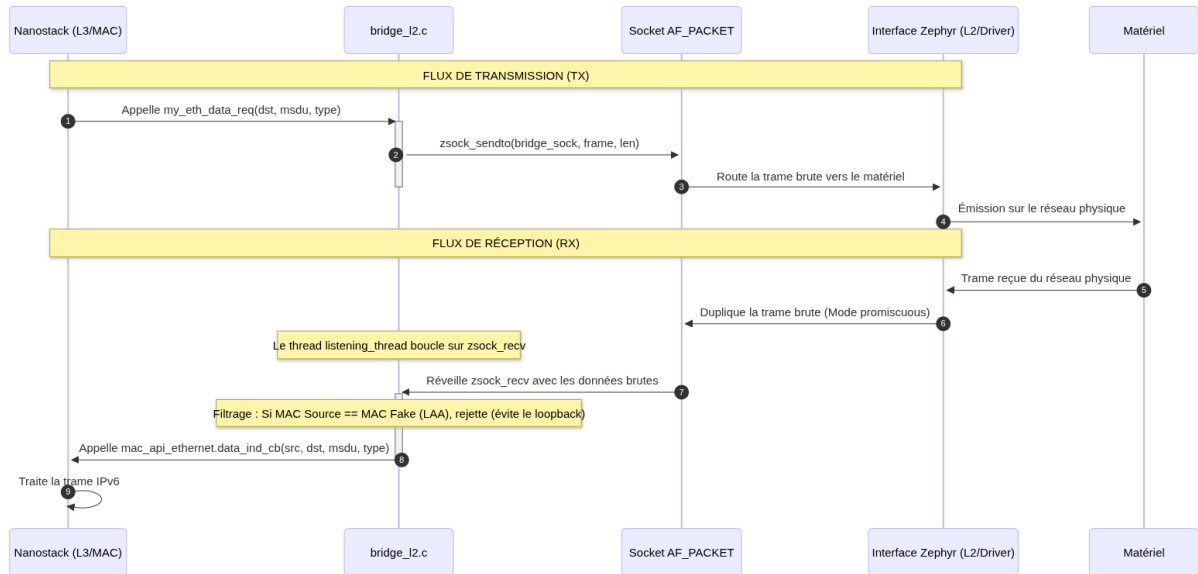


Fig. 5. – Architecture et flux de données à travers le pont logiciel bridge_l2.c

Pour pallier les limitations rencontrées lors de l'implémentation d'une couche L2 personnalisée, le choix s'est porté sur l'utilisation des sockets AF_PACKET. Ce mécanisme permet d'intercepter les trames Ethernet directement au niveau de la couche de liaison de données, en contournant les piles réseau supérieures de Zephyr. Cette approche offre une grande maîtrise sur les flux de données :

- **Flux entrant (Rx)** : Les trames Ethernet sont capturées dès leur réception par le pilote matériel, puis injectées directement dans la Nanostack. Ce processus garantit que la pile réseau traite les données brutes sans interférence du noyau.
- **Flux sortant (Tx)** : Les trames générées par la Nanostack sont récupérées avant leur traitement par le système et transmises au driver physique de Zephyr. Cela permet de conserver une totale autonomie sur la construction et l'encapsulation des paquets.

L'avantage majeur de cette méthode réside dans sa capacité à traiter les trames de niveau 2 de manière transparente. En utilisant les sockets AF_PACKET, le pont bénéficie de la sûreté des drivers de Zephyr tout en conservant le contrôle complet sur l'intégralité du cycle de vie des paquets au sein de la Nanostack. C'est une solution résiliente qui assure une communication fluide entre la pile réseau et le matériel, sans compromettre l'intégrité du noyau. Cette architecture, bien qu'extrêmement stable et flexible, introduit une légère surcharge au niveau du traitement des paquets, liée au passage par les sockets AF_PACKET et au contexte de commutation entre la pile réseau et le noyau. Toutefois, après analyse, cette perte de

performance s'avère négligeable au regard des gains en termes de fiabilité et de maintenabilité. Elle n'impacte en rien la stabilité globale du système.

7.4.3.1 Gestion de la virtualisation d'adresse MAC et du filtrage

La gestion de l'adressage MAC a soulevé une problématique d'unicité sur le support physique. Afin d'éviter tout conflit de boucles réseau ou de collision de paquets, une distinction claire a été implémentée entre l'adresse MAC réelle de l'interface gérée par **Zephyr** et une adresse MAC virtuelle dédiée à la **Nanostack**. En manipulant le premier octet de l'adresse, comme illustré dans le Listing 16, le premier octet de l'adresse est modifié afin de forcer l'activation du bit d'administration locale (**Locally Administered Address**). Cette pratique standard permet d'indiquer que cette adresse n'est pas une adresse universelle définie par le constructeur, mais une entité réseau spécifique gérée logiciellement.

```
// Récupération de la MAC réelle de l'interface
struct net_linkaddr *link_addr = net_if_get_link_addr(iface);
memcpy(nanostack_mac_addr, link_addr->addr, 6);
// Forçage du bit d'administration locale (LAA)
// Cela distingue l'entité Nanostack de l'interface Zephyr
nanostack_mac_addr[0] |= 0x02;
```

Listing 16. – Virtualisation de l'adresse MAC via le bit d'administration locale

Cette séparation est indispensable pour que la **Nanostack** opère comme un nœud distinct sur le réseau physique, alors même qu'elle partage le même contrôleur Ethernet que le système hôte. Trois aspects techniques critiques complètent cette implémentation au sein de `bridge_l2.c` :

1. **Le Mode Promiscuous** : L'appel à `net_if_set_promisc(iface)` force le contrôleur Ethernet physique à accepter tous les paquets du média, y compris ceux destinés à notre MAC virtuelle. Sans cela, le filtre matériel du contrôleur rejetterait immédiatement les trames destinées à la Nanostack.
2. **Le filtrage du Loopback local** : Lors de l'envoi de trames via le socket brut `AF_PACKET`, Zephyr copie par défaut ces trames sur tous les sockets écouteurs de cette interface. Le thread de réception (`listening_thread`) doit donc vérifier si la MAC source de la trame reçue correspond à la MAC virtuelle (`nanostack_mac_addr`). Si c'est le cas, la trame est ignorée pour couper court à une boucle de rétroaction infinie.
3. **L'IID et l'Autoconfiguration IPv6** : L'adresse MAC virtuelle est passée à la fonction `my_iid64_get` afin de calculer l'identifiant d'interface (IID) IPv6 conforme aux RFC de routage. Cet IID est forcé dans les champs `iid_eui64` et `iid_slaac` de la Nanostack. Les adresses

IPv6 générées par auto-configuration (SLAAC) dérivent ainsi proprement de la MAC virtuelle et n'entrent pas en collision avec celles de Zephyr.

7.4.4 Intégration de la liaison Radio AF_PACKET (bridge_l2_radio.c)

Contrairement à la liaison Ethernet où la couche de liaison L2 de Zephyr gère la quasi-totalité des opérations d'encapsulation de manière transparente, l'intégration d'une radio IEEE 802.15.4 (Sub-GHz) requiert un contrôle beaucoup plus fin de la couche physique (L1) par la pile Nanostack. La stack doit pouvoir piloter l'état de la radio, initier l'écoute du canal (CCA - **C**lear **C**hannel **A**ssessment), changer de fréquence ou encore obtenir les métriques de qualité de lien (LQI et RSSI).

Pour répondre à ces exigences, le pont `bridge_l2_radio.c` enregistre un pilote physique virtuel auprès de la Nanostack via la structure `phy_device_driver_s`. Les callbacks de cette structure redirigent les requêtes matérielles de la Nanostack vers l'API radio native de Zephyr (`ieee802154_radio_api`), tout en utilisant des sockets bruts AF_PACKET pour le transit des trames de données.

7.4.4.1 Architecture de séparation des Sockets (Socket Split) et Thread RX

L'implémentation initiale utilisant un unique socket bidirectionnel sur la file de travail système de Zephyr (**system work queue**) provoquait des interblocages (**deadlocks**) et des pertes de paquets lors d'un trafic bidirectionnel intense.

Pour y pallier, une architecture de séparation des descripteurs de sockets a été mise en place (Listing 17) :

1. Un socket dédié exclusivement à l'émission : `radio_bridge_sock_tx`.
2. Un socket dédié exclusivement à la réception : `radio_bridge_sock_rx`.

```
// Création du socket d'émission (TX)
radio_bridge_sock_tx = zsock_socket(AF_PACKET, SOCK_RAW, htons(ETH_P_ALL));
zsock_bind(radio_bridge_sock_tx, (struct sockaddr *)&sll, sizeof(sll));

// Création du socket de réception (RX)
radio_bridge_sock_rx = zsock_socket(AF_PACKET, SOCK_RAW, htons(ETH_P_ALL));
zsock_bind(radio_bridge_sock_rx, (struct sockaddr *)&sll, sizeof(sll));
```

Listing 17. – Initialisation et séparation des sockets TX et RX

Cette séparation s'accompagne d'un thread d'écoute dédié à la réception (`radio_rx_thread`), s'exécutant de manière asynchrone avec une priorité temps réel. Ce thread boucle sur un appel bloquant à `zsock_recvfrom` pour intercepter les trames entrantes et les réinjecter immédiatement dans la Nanostack via le callback `phy_rx_cb` (Listing 18).

```
static void radio_rx_thread(void *p1, void *p2, void *p3) {
    static uint8_t rx_buf[2048];
    struct sockaddr_ll src_addr;
    socklen_t addr_len;

    while (1) {
        addr_len = sizeof(src_addr);
        int len = zsock_recvfrom(radio_bridge_sock_rx, rx_buf, sizeof(rx_buf), 0,
                                (struct sockaddr *)&src_addr, &addr_len);

        // Élimination des trames émises par notre propre nœud (loopback)
        if (src_addr.sll_pkttype == PACKET_OUTGOING) {
            continue;
        }

        // Transmission de la trame brute directement à la Nanostack
        if (radio_driver.phy_rx_cb && radio_phy_id >= 0) {
            nanostack_lock();
            radio_driver.phy_rx_cb(rx_buf, len, 255, -50, radio_phy_id);
            nanostack_unlock();
        }
    }
}
```

Listing 18. – Thread de réception radio asynchrone et filtrage

Cette structure asynchrone évite que le traitement d’une réception ne vienne bloquer la boucle d’émission de paquets ou le cœur d’ordonnancement de la Nanostack.

7.4.4.2 Cycle de transmission et émulation de fin d’émission (TX State Machine)

Lors de l’émission d’un paquet, la Nanostack appelle la fonction `radio_tx`. Afin de respecter le protocole CSMA-CA (évitement de collision) sans bloquer le thread appelant, le pont utilise une machine à états asynchrone s’appuyant sur le gestionnaire de tâches de Zephyr (**K_WORK**) et des timers.

Le flux d’émission se déroule comme suit :

1. `radio_tx` copie le paquet dans un tampon temporaire `pending_tx_buf` et planifie la tâche `tx_prepare_work`.
2. Le gestionnaire `tx_prepare_work_handler` interroge la pile pour s’assurer que le canal est libre (`PHY_LINK_CCA_PREPARE`).
3. Si le canal est libre, le paquet est envoyé sur le média via le socket `radio_bridge_sock_tx`.

4. Le temps de transmission physique est estimé en fonction du nombre d'octets et du débit de modulation configuré (`radio_channel_config.datarate`). Un timer de fin de transmission (`tx_done_timer`) est alors démarré avec cette durée calculée.
5. À l'expiration du timer, la fonction de rappel `phy_tx_done_cb` est appelée avec le statut `PHY_LINK_TX_SUCCESS`, indiquant à la Nanostack qu'elle peut libérer ses tampons et planifier le paquet suivant.

7.4.4.3 Limitation des fréquences et contournement du pilote Texas Instruments CC1352

Un défi majeur est apparu lors des phases de test avec les modules Texas Instruments CC1352P7. Le protocole Wi-SUN en Europe est conçu pour s'étendre sur la bande 863 MHz découpée en 35 canaux (de 0 à 34). Lors de la phase de découverte du réseau (**PAN Discovery**), la Nanostack tente d'effectuer des sauts de fréquence automatiques sur l'ensemble de ces canaux.

Cependant, l'examen approfondi du code source du pilote d'origine fourni par Texas Instruments dans Zephyr (`zephyr/drivers/ieee802154/ieee802154_cc13xx_cc26xx_subg.c`) révèle une limitation logicielle stricte. La fonction chargée de calculer la fréquence à configurer sur la puce radio n'autorise que le canal 0 (868.3 MHz) et rejette systématiquement les canaux supérieurs à 10, comme le montre le Listing 19.

```
static inline int drv_channel_frequency(uint16_t channel, uint16_t *frequency, uint16_t *fractFreq)
{
    /* ... */
    if (channel == 0) {
        *frequency = 868;
        *fractFreq = 0x4ccd; // Fréquence de 868.3 MHz
    } else if (channel <= 10) {
        *frequency = 906 + 2 * (channel - 1); // Bande US 915 MHz
        *fractFreq = 0;
    } else {
        *frequency = 0;
        *fractFreq = 0;
        return channel <= 26 ? -ENOTSUP : -EINVAL; // Erreur renvoyée !
    }
    /* TODO: This incorrectly mixes up legacy BPSK SubGHz PHY channel page
     * zero frequency calculation with SUN FSK operating mode #3 PHY radio... */
    return 0;
}
```

Listing 19. – Limitation des canaux dans le pilote de périphérique TI CC1352 de Zephyr

Comme l'indique explicitement le commentaire TODO laissé par les développeurs du pilote, le calcul des fréquences mélange par erreur la configuration historique BPSK de la page 0 avec les nouveaux paramètres requis par la modulation SUN FSK (utilisée par Wi-SUN). Le pilote est donc incapable de calculer dynamiquement la fréquence pour un canal Wi-SUN valide (comme le canal 12 ou 24 sur la page de canaux 9).

Pour contourner cette limitation du constructeur sans avoir à réécrire intégralement le pilote de périphérique de la puce CC1352, un filtrage logique applicatif a été implémenté via l'API de gestion Wi-SUN. En configurant un masque de canaux restrictif à l'aide de la fonction `ws_management_channel_mask_set` (en forçant le bit du canal 0 à 1 et tous les autres à 0), la Nanostack a été contrainte à rester exclusivement sur le canal 0.

Cette approche a permis de valider notre objectif premier : apporter la preuve de concept que notre pont radio fonctionne, et que les nœuds parviennent à s'échanger des trames physiques sur l'unique fréquence opérationnelle de 868.3 MHz. Néanmoins, cette configuration figeant la communication sur un canal unique empêche l'activation du mécanisme de saut de fréquence (FHSS) obligatoire pour la certification Wi-SUN. Bien que la communication radio soit fonctionnelle et démontrée, le système ne peut pas dans cet état établir un véritable réseau Wi-SUN conforme aux spécifications standard.

7.4.4.4 Étude d'une alternative : Le pont radio direct et ses limitations

Dans l'optique d'optimiser au maximum la latence de transmission et d'éliminer la surcharge induite par le passage par les sockets du noyau, une variante nommée « pont direct » (`bridge_l2_radio_direct.c`) a été conçue et expérimentée.

L'idée théorique était de court-circuiter entièrement la pile réseau de Zephyr :

- **En réception (Rx)** : Enregistrer un filtre réseau synchrone (`net_pkt_filter`) pour capturer la trame dès sa réception par le pilote et la transmettre directement à la Nanostack, en demandant au noyau de détruire le paquet d'origine pour éviter un double traitement.
- **En émission (Tx)** : Allouer un paquet brut et appeler directement la fonction d'émission physique du transceiver (`radio_api->tx`) exposée par le pilote matériel de Zephyr, sans passer par les abstractions supérieures.

Bien que séduisante, cette implémentation s'est avérée instable et a conduit à des dysfonctionnements critiques. L'analyse de ces échecs a mis en lumière trois contraintes architecturales majeures de Zephyr :

1. **Le problème de la fragmentation mémoire (RX)** : Les paquets réseau dans Zephyr (`net_pkt`) sont stockés sous forme de listes chaînées de fragments de tampons (`net_buf`). Dans l'implémentation directe, la lecture brute des données via `buf->data` ne lisait que le premier

fragment, alors que la taille passée correspondait à la trame entière. Cela provoquait des lectures hors limites de la mémoire (**buffer overflow** en lecture), menant à des corruptions de données et des plantages système (**Kernel Panic**). Pour résoudre cela proprement, il aurait fallu réimplémenter une copie linéaire complète via `net_pkt_read`, annulant le gain de performance espéré.

2. **L'inversion de contexte et les interblocages (Deadlocks)** : Les règles de filtrage de paquets (`net_pkt_filter`) sont exécutées directement dans le contexte d'interruption (ISR) du pilote matériel ou dans le thread réseau à haute priorité de Zephyr (`net_rx`). Appeler le callback de traitement de la Nanostack dans ce contexte forçait le processeur à exécuter la logique de routage L3 complexe au milieu de l'ISR. De plus, la nécessité d'acquérir le verrou `nanostack_lock()` dans ce contexte d'interruption créait des conflits d'exclusion mutuelle avec les autres threads, provoquant des interblocages (**deadlocks**) complets du système.
3. **L'absence de synchronisation de l'émetteur (TX)** : L'API physique `radio_api->tx` initie l'envoi de manière asynchrone au niveau matériel. Signaler immédiatement une réussite de transmission (`PHY_LINK_TX_SUCCESS`) à la Nanostack dès le retour de cette fonction — sans attendre le déroulement du protocole CSMA-CA ni la réception de l'acquittement physique (ACK) par la puce radio — faussait complètement la logique de retransmission de la pile réseau, causant d'importantes pertes de paquets.

Cette tentative infructueuse a ainsi démontré qu'un pont direct synchrone brise l'isolation des contextes de Zephyr. Cela a pleinement validé le choix de notre architecture finale basée sur les sockets `AF_PACKET` avec séparation des canaux d'émission et de réception, qui délègue proprement la gestion des files d'attente et le changement de contexte de thread au système d'exploitation.

7.4.5 Boucle d'événements et ordonnancement (`event_loop.c`)

La pile réseau Nanostack est conçue selon un modèle asynchrone événementiel. Elle n'exécute pas de threads en interne, mais s'appuie sur son propre ordonnanceur non préemptif coopératif (`eventOS_scheduler`). Cet ordonnanceur empile les tâches (timers, réceptions de trames, calculs de routes) dans une file d'attente globale et attend qu'un thread hôte les exécute séquentiellement.

Afin d'intégrer ce modèle dans l'architecture préemptive de Zephyr, une boucle d'événements dédiée a été implémentée au sein du fichier `event_loop.c` (Listing 20).

```

static void nanostack_thread_entry(void *p1, void *p2, void *p3){
    while (1){
        // Exécution verrouillée de l'ordonnanceur coopératif
        nanostack_lock();
        eventOS_scheduler_run_until_idle();
        nanostack_unlock();

        // Mise en veille optimale jusqu'au prochain événement matériel
        k_sem_take(&ns_event_sem, K_FOREVER);
    }
}

// Déclaration du thread avec priorité temps réel préemptive de 0
K_THREAD_DEFINE(nanostack_thread_id, 8192, nanostack_thread_entry,
                NULL, NULL, NULL, 0, 0, -1);

```

Listing 20. – Boucle d'événements et thread dédié à la Nanostack

Cette implémentation repose sur trois concepts clés :

1. **Un thread[16] dédié de haute priorité** : La Nanostack traite des protocoles réseau sensibles au facteur temps (comme le saut de fréquence en Wi-SUN). Le thread `nanostack_thread_id` est donc instancié avec une priorité préemptive de 0 (la plus haute priorité pour les threads applicatifs de Zephyr) et une pile confortable de 8 Ko pour éviter tout débordement lors des calculs cryptographiques ou de routage.
2. **L'accès exclusif à la pile réseau (`nanostack_lock`)** : Afin d'éviter les accès concurrents destructeurs entre les threads de réception réseau de Zephyr et la boucle d'ordonnancement de la Nanostack, l'appel à `eventOS_scheduler_run_until_idle()` est entièrement encapsulé dans une section d'exclusion mutuelle (**Mutex**).
3. **Mise en veille et optimisation énergétique (`K_FOREVER`)** : Une fois que l'ordonnanceur a traité tous les événements en attente et est devenu inactif (**idle**), le thread s'endort sur le sémaphore `ns_event_sem`.
 - **Choix du `K_FOREVER`** : Pendant la phase de développement, un timeout temporaire de 1 ms (`K_MSEC(1)`) avait été mis en place comme sécurité (**fallback**) pour forcer le réveil périodique de la pile en cas de signal manqué. Une fois le portage fiabilisé, ce timeout a été remplacé par `K_FOREVER`. La Nanostack notifiant systématiquement chaque nouvel événement (expiration de timer, réception de paquets) via l'appel `eventOS_scheduler_signal()`, le réveil est garanti à chaque événement réel.
 - **Impact sur la consommation** : L'utilisation de `K_FOREVER` évite 1000 réveils de contexte inutiles par seconde. Lorsque la file d'attente est vide, le microcontrôleur peut basculer immédiatement sur son thread d'inactivité (**Idle Thread**), coupant les

horloges internes du processeur via l'instruction WFI (Wait For Interrupt) et optimisant ainsi grandement l'autonomie sur batterie.

8 Validation et tests de fonctionnement

Pour valider l'intégrité de la pile réseau Nanostack et tester l'ensemble des scénarios de communication envisagés, trois applications de test distinctes ont été développées. Elles se divisent en deux axes de validation complémentaires :

1. **L'axe filaire (Ethernet)** : Une application de validation conçue pour tester le mécanisme de pontage L2 par socket brute sur un support Ethernet classique. Ce scénario simplifié permet de valider le comportement de l'ordonnanceur, l'intégration de la couche de transport UDP, ainsi que la configuration dynamique des adresses IPv6 (SLAAC) sans les perturbations inhérentes aux liaisons radio.
2. **L'axe sans fil (Wi-SUN)** : Un ensemble composé de deux applications (un routeur de bordure **Border Router** et un nœud périphérique **End Device**). Ce système complet permet de valider le pont radio 802.15.4, d'observer la phase de découverte de réseau (PAN Discovery) et de confirmer la capacité de la pile à acheminer des paquets IPv6 sur un média physique partagé.

Ces applications font office de démonstrateurs fonctionnels mais servent également de cibles de compilation de référence pour valider l'intégration du module au système de build de Zephyr.

8.1 Validation de la communication filaire (Ethernet)

Le premier programme de test, implémenté dans le module `app_ethernet`, a pour objectif de valider le comportement de base de la **Nanostack** et du pont logiciel Ethernet (`bridge_l2.c`) sur un canal physique stable et exempt de pertes (liaison filaire). La validation s'est articulée autour de deux phases de tests successives.

8.1.1 Validation de la connectivité Link-Local

La première étape de validation a consisté à vérifier la capacité de la **Nanostack** à établir une communication de base en mode **Link-Local** sur le support filaire. L'objectif était de confirmer que le pont logiciel transmettait correctement les paquets ICMPv6 de découverte de voisins (**Neighbor Discovery**) sans altération, permettant ainsi une autoconfiguration IPv6 fonctionnelle.

Le protocole **Neighbor Discovery** est critique en IPv6 : il permet aux interfaces réseau de découvrir les nœuds adjacents et de résoudre leurs adresses matérielles. En utilisant une interface Ethernet, ce mécanisme a été isolé afin de tester la transparence du pont :

- **Initialisation** : Dès l'activation de l'interface, la **Nanostack** génère une requête de sollicitation de voisin (**Neighbor Solicitation**) pour résoudre l'adresse IP cible.
- **Transmission** : Le pont intercepte cette requête, l'encapsule dans une trame Ethernet, et la transmet au driver **Zephyr** via le socket **AF_PACKET**.
- **Réponse** : Le nœud distant répond avec une publicité de voisin (**Neighbor Advertisement**). Cette trame suit le chemin inverse, transitant par le pont pour être injectée dans la pile **Nanostack**.

No.	Time	Source	Destination	Protocol	Length	Info
1	0.000000	fe80::1807:384a:90f4:a667	fe80::1807:384a:90f4:a667	ICMPv6	128	Echo (ping) request id=0x0000, seq=1, hop limit=64 (reply in 3)
2	0.000000	fe80::1807:384a:90f4:a667	fe80::1807:384a:90f4:a667	ICMPv6	70	Router Solicitation From fe80::1807:384a:90f4:a667
3	0.000000	fe80::1807:384a:90f4:a667	fe80::1807:384a:90f4:a667	ICMPv6	128	Echo (ping) reply id=0x0000, seq=1, hop limit=64 (request in 1)
4	0.000100	fe80::1807:384a:90f4:a667	fe80::1807:384a:90f4:a667	ICMPv6	128	Echo (ping) request id=0x0000, seq=2, hop limit=64 (reply in 5)
5	0.000100	fe80::1807:384a:90f4:a667	fe80::1807:384a:90f4:a667	ICMPv6	128	Echo (ping) request id=0x0000, seq=2, hop limit=64 (request in 4)
6	0.000200	fe80::1807:384a:90f4:a667	fe80::1807:384a:90f4:a667	ICMPv6	128	Echo (ping) request id=0x0000, seq=3, hop limit=64 (reply in 7)
7	0.000200	fe80::1807:384a:90f4:a667	fe80::1807:384a:90f4:a667	ICMPv6	128	Echo (ping) request id=0x0000, seq=3, hop limit=64 (request in 6)
8	0.000300	fe80::1807:384a:90f4:a667	fe80::1807:384a:90f4:a667	ICMPv6	128	Echo (ping) request id=0x0000, seq=4, hop limit=64 (reply in 9)
9	0.000300	fe80::1807:384a:90f4:a667	fe80::1807:384a:90f4:a667	ICMPv6	128	Echo (ping) request id=0x0000, seq=4, hop limit=64 (request in 8)
10	0.000400	fe80::1807:384a:90f4:a667	fe80::1807:384a:90f4:a667	ICMPv6	128	Echo (ping) request id=0x0000, seq=5, hop limit=64 (reply in 11)
11	0.000400	fe80::1807:384a:90f4:a667	fe80::1807:384a:90f4:a667	ICMPv6	128	Echo (ping) request id=0x0000, seq=5, hop limit=64 (request in 10)
12	0.000500	fe80::1807:384a:90f4:a667	fe80::1807:384a:90f4:a667	ICMPv6	128	Echo (ping) request id=0x0000, seq=6, hop limit=64 (reply in 13)
13	0.000500	fe80::1807:384a:90f4:a667	fe80::1807:384a:90f4:a667	ICMPv6	128	Echo (ping) request id=0x0000, seq=6, hop limit=64 (request in 12)
14	0.000600	fe80::1807:384a:90f4:a667	fe80::1807:384a:90f4:a667	ICMPv6	128	Echo (ping) request id=0x0000, seq=7, hop limit=64 (reply in 15)
15	0.000600	fe80::1807:384a:90f4:a667	fe80::1807:384a:90f4:a667	ICMPv6	86	Neighbor Solicitation for fe80::1807:384a:90f4:a667 from fe80::1807:384a:90f4:a667
16	0.000700	fe80::1807:384a:90f4:a667	fe80::1807:384a:90f4:a667	ICMPv6	70	Neighbor Advertisement From fe80::1807:384a:90f4:a667
17	0.000700	fe80::1807:384a:90f4:a667	fe80::1807:384a:90f4:a667	ICMPv6	128	Echo (ping) reply id=0x0000, seq=7, hop limit=64 (request in 14)
18	0.000800	fe80::1807:384a:90f4:a667	fe80::1807:384a:90f4:a667	ICMPv6	128	Echo (ping) request id=0x0000, seq=8, hop limit=64 (reply in 18)
19	0.000800	fe80::1807:384a:90f4:a667	fe80::1807:384a:90f4:a667	ICMPv6	128	Echo (ping) request id=0x0000, seq=8, hop limit=64 (request in 17)
20	0.000900	fe80::1807:384a:90f4:a667	fe80::1807:384a:90f4:a667	ICMPv6	128	Echo (ping) request id=0x0000, seq=9, hop limit=64 (reply in 21)
21	0.000900	fe80::1807:384a:90f4:a667	fe80::1807:384a:90f4:a667	ICMPv6	128	Echo (ping) request id=0x0000, seq=9, hop limit=64 (request in 20)
22	0.001000	fe80::1807:384a:90f4:a667	fe80::1807:384a:90f4:a667	ICMPv6	128	Echo (ping) request id=0x0000, seq=10, hop limit=64 (reply in 23)
23	0.001000	fe80::1807:384a:90f4:a667	fe80::1807:384a:90f4:a667	ICMPv6	128	Echo (ping) request id=0x0000, seq=10, hop limit=64 (request in 22)
24	0.001100	fe80::1807:384a:90f4:a667	fe80::1807:384a:90f4:a667	ICMPv6	128	Echo (ping) request id=0x0000, seq=11, hop limit=64 (reply in 25)
25	0.001100	fe80::1807:384a:90f4:a667	fe80::1807:384a:90f4:a667	ICMPv6	70	Router Solicitation From fe80::1807:384a:90f4:a667
26	0.001200	fe80::1807:384a:90f4:a667	fe80::1807:384a:90f4:a667	ICMPv6	128	Echo (ping) request id=0x0000, seq=11, hop limit=64 (request in 24)
27	0.001200	fe80::1807:384a:90f4:a667	fe80::1807:384a:90f4:a667	ICMPv6	128	Echo (ping) request id=0x0000, seq=12, hop limit=64 (reply in 28)

Fig. 6. – Processus de découverte de voisins (NDP) à travers le pont logiciel

Le succès de cette phase est validé par la réception effective de réponses ICMPv6. La capture réseau Fig. 6 montre que les paquets transitent avec les adresses MAC virtuelles configurées précédemment. Cette validation prouve que la logique de routage de la **Nanostack** communique correctement avec le support physique, et que les mécanismes de bas niveau (L2) assurés par le pont ne créent aucune latence bloquante pour les protocoles de découverte IPv6.

8.1.2 Phase 2: Validation de l'auto-configuration et de la couche IP

La seconde étape de validation a pour objectif de confirmer la pleine opérabilité de la pile réseau de la **Nanostack** au-dessus du pont logiciel. Il s'agit ici de vérifier que l'encapsulation des trames est correctement interprétée et que les mécanismes d'adressage IPv6 sont fonctionnels.

- Le serveur écoute sur le port 12345.
- Un gestionnaire d'événements (`app_tasklet_handler`) intercepte la connexion d'un client externe (`SOCKET_INCOMING_CONNECTION`) et accepte la liaison via la fonction native `socket_accept`.
- Un timer logique de la **Nanostack** se déclenche toutes les secondes pour incrémenter un compteur et envoyer la chaîne de caractères "Compteur: X\n" au client TCP connecté (Listing Listing 21).

```
void app_tasklet_handler(arm_event_s *event) {
    if (event->event_type == ARM_LIB_TASKLET_INIT_EVENT) {
        app_tcp_listener = socket_open(SOCKET_TCP, 12345, app_tcp_listener_callback);
        socket_listen(app_tcp_listener, 1);
        eventOS_event_timer_request(1, 102, app_tasklet_id, 1000);
    } else if (event->event_type == 102) { // Événement timer (1s)
        send_counter++;
        if (app_tcp_client_socket >= 0) {
            char send_buf[64];
            int len = snprintf(send_buf, sizeof(send_buf), "Compteur: %u\n", send_counter);
            socket_send(app_tcp_client_socket, send_buf, len);
        }
        eventOS_event_timer_request(1, 102, app_tasklet_id, 1000); // Relance du timer
    }
}
```

Listing 21. – Gestion de la connexion TCP et envoi du compteur dans le Tasklet

Ce test a été validé en se connectant à la carte depuis le PC hôte à l'aide de l'utilitaire `netcat` (`nc -6 <adresse_ip_carte> 12345`). La réception fluide du flux de compteurs a prouvé que la gestion des fenêtres de réception, des acquittements TCP et de l'ordonnancement des sockets de la **Nanostack** fonctionne de concert avec le système de threads de Zephyr.

8.1.4 Analyse de la latence réseau

Afin de quantifier l'impact du pontage logiciel sur la performance globale du système, une série de tests de latence basés sur l'envoi de 100 requêtes ICMPv6 (`ping6`) a été réalisée. Ces mesures permettent d'évaluer le temps de traitement induit par le passage à travers le socket `AF_PACKET` et la pile **Nanostack** dans différents scénarios de configuration.

Les résultats synthétisés dans le Tableau Tableau 3 mettent en évidence une excellente réactivité en conditions nominales. En communication **Link-Local** ou via un routeur, la latence moyenne demeure inférieure à 1 ms, confirmant que le pontage logiciel n'introduit qu'une surcharge négligeable.

En revanche, l'activation des traces de débogage de la pile (`mbed_trace` redirigé vers le port série via la fonction `printf`) révèle une augmentation significative de la latence, passant à environ 60 ms. Cette hausse est due au caractère synchrone de l'affichage série sous Zephyr : à un débit de 115 200 bauds, l'envoi de chaque caractère bloque activement le processeur pendant environ 87 μ s. L'écriture de plusieurs lignes de log détaillées lors du traitement d'une trame accapare ainsi le temps CPU, ce qui retarde d'autant l'exécution de la boucle d'événements de la Nanostack et augmente artificiellement le temps de réponse aux requêtes d'écho.

Scénario	Min (ms)	Moy (ms)	Max (ms)	Écart-type (ms)
Link-Local	0.58	0.68	1.20	0.08
Via Routeur	0.63	0.71	1.08	0.08
Routeur (Traces actives)	59.45	59.99	87.68	3.15

Tableau 3. – Analyse des performances de latence ICMPv6

Ces mesures démontrent la viabilité de l'architecture pour des applications temps-réel standards. La stabilité des écarts-types en conditions nominales prouve que le déterminisme du système est préservé, rendant cette implémentation robuste pour un usage en milieu industriel.

8.2 Validation de la communication sans fil (Wi-SUN / 802.15.4)

La validation sans fil s'appuie sur deux programmes distincts fonctionnant en miroir : le routeur de bordure (`app_border_router`) et le nœud périphérique (`app_device_node`). Ils s'exécutent sur deux cartes distinctes équipées de puces radio TI CC1352P7.

8.2.1 Le Routeur de Bordure (`app_border_router`)

Le routeur de bordure (6LBR - **IPv6 Low-power Border Router**) est le nœud d'infrastructure central du réseau Wi-SUN. Il coordonne le PAN, gère l'authentification des nœuds entrants et assure le routage entre le domaine radio 802.15.4 et le backbone Ethernet.

L'initialisation de l'application s'effectue dans sa tâche principale (`app_tasklet_handler`) :

1. **Configuration du bootstrap** : L'interface est déclarée comme `NET_6LOWPAN_BORDER_ROUTER` sous le profil de réseau `NET_6LOWPAN_WS` (Wi-SUN).

2. **Authentication (EAP-TLS)** : Le routeur charge le certificat de l'autorité racine (CA) et son propre certificat de serveur de test afin d'agir comme authentificateur.
3. **Paramétrage physique et réglementaire** : Le routeur est configuré pour la bande européenne Sub-GHz (REG_DOMAIN_EU) en mode FSK à 50 kbps (Operating Mode 1a).
4. **Restriction au canal 0** : Afin de contourner la limitation physique du pilote Zephyr, les mécanismes de saut de fréquence (FHSS) sont débrayés. Les canaux d'émission unicast et broadcast sont fixés sur le canal 0 (868.3 MHz) et le masque de canaux Wi-SUN est restreint à cette seule fréquence.
5. **Démarrage du service BBR** : Le routeur configure son PAN ID (0x1234), active la diffusion de sa route par défaut via les paramètres Backbone Router (ws_bbr_configure) et démarre l'interface.

8.2.2 Le Nœud Périphérique (app_device_node)

Le nœud périphérique (6LN - **IPv6 Low-power Node**) est un appareil final destiné à rejoindre le PAN créé par le routeur de bordure afin d'envoyer ses données capteurs ou télémétriques.

L'initialisation de son tasklet de contrôle (app_tasklet_handler) est calquée sur celle du routeur, garantissant l'alignement des paramètres réseau (Listing Listing 22) :

- **Rôle de nœud** : Il est configuré comme simple routeur/hôte Wi-SUN (NET_6LOWPAN_ROUTER).
- **Certificats clients** : Il charge le certificat racine de confiance (CA) et son propre certificat client Wi-SUN (WISUN_CLIENT_CERTIFICATE) pour s'authentifier auprès du routeur.
- **Alignement physique** : Les paramètres de domaine réglementaire (Europe, 50 kbps FSK) et la restriction au canal fixe 0 sont identiques à ceux du routeur.

```

void app_tasklet_handler(arm_event_s *event) {
    if (event->event_type == ARM_LIB_TASKLET_INIT_EVENT) {
        // Configuration du rôle de routeur/hôte Wi-SUN
        arm_nwk_interface_configure_6lowpan_bootstrap_set(
            nwk_interface_id, NET_6LOWPAN_ROUTER, NET_6LOWPAN_WS);

        // Ajout des certificats du Suppliquant (EAP-TLS client)
        arm_network_trusted_certificate_add(&trusted_cert);
        arm_network_own_certificate_add(&own_cert);

        // Alignement réglementaire et forçage du canal 0 (868.3 MHz)
        ws_management_node_init(nwk_interface_id, REG_DOMAIN_EU, "Wi-SUN-Network", fhss_timer);
        ws_management_regulatory_domain_set(nwk_interface_id, REG_DOMAIN_EU, 2, OPERATING_MODE_1a);

        uint32_t channel_mask[8] = {1}; // Canal 0 uniquement
        ws_management_channel_mask_set(nwk_interface_id, channel_mask);
        ws_management_fhss_unicast_channel_function_configure(nwk_interface_id, 0, 0, 0);

        // Démarrage de l'interface
        arm_nwk_interface_up(nwk_interface_id);
    }
}

```

Listing 22. – Configuration Wi-SUN et restriction de canal sur le nœud périphérique

Une fois démarré, le nœud périphérique ouvre son propre socket UDP d'émission et entre dans une phase d'écoute active du média (balayage réseau) à la recherche de trames d'annonce PAN (**PAN Advertisements**) émises par le routeur de bordure afin de démarrer sa procédure d'association.

8.2.3 Validation de la couche de pontage radio

Bien que l'intégration complète du protocole Wi-SUN se soit heurtée aux limitations du pilote radio natif, la validation de la couche d'adaptation radio a pu être réalisée avec succès. L'objectif était de confirmer que le pont radio (`bridge_l2_radio.c` adapté) est capable de gérer le flux de données bidirectionnel entre la pile radio et le noyau.

Les tests effectués confirment la robustesse de l'interfaçage : nous observons un trafic TX/RX conforme aux attentes, prouvant que le passage de la trame radio vers le socket `AF_PACKET` et sa réinjection dans la pile sont opérationnels. Comme illustré dans la Fig. 8, la pile radio détecte et traite les trames entrantes, tandis que les requêtes d'émission sont correctement transmises au driver physique.

```
[DEVICE] [MODE] Ajout certificat système de confiance: 0
[DEVICE] [MODE] Ajout certificat client propre: 0
[DEVICE] [MODE] Adresse FHSS Timer: 0x20000690
[DEVICE] [MODE] ws_management_node_init: 0
[DEVICE] [MODE] ws_management_regulatory_domain_set: 0
[DEVICE] [MODE] ws_management_channel_mask_set: 0
[DEVICE] [MODE] ws_management_fhss_unicast_channel_function_configure: 0
[DEVICE] [MODE] ws_management_fhss_broadcast_channel_function_configure: 0
[INFO] [wsbs]: MAC address: 00:12:4b:00:14:f9:68:ff
[INFO] [addr]: Address added to IF 1: fe80::212:4b00:14f9:68ff
[DBG] [rout]: On-Link (net 128)
[DBG] [rout]: On-Link (net 192)
[INFO] [antr]: Monitor rate limit incoming packets at:1024
[INFO] [antr]: Monitor set high:62213, critical:64178 total:65488
[DEVICE] [MODE] ara_nwk_interface_up: 0
[INFO] [wsbs]: Discovery start
[INFO] [wsbs]: Wi-SUN packet congestion minTh 32, maxTh 65, drop probability 10 weight 32, Packet/Seconds 2
[INFO] [nlme]: RF config update
[INFO] [nlme]: Frequency(ch0): 863100000Hz
[INFO] [nlme]: Channel spacing: 1000000Hz
[INFO] [nlme]: Datarate: 50000bps
[INFO] [nlme]: Number of channels: 69
[INFO] [nlme]: Modulation: 4
[INFO] [nlme]: Modulation index: 0
[INFO] [nlme]: FEC: 0
[INFO] [nlme]: OFDM MCS: 0
[INFO] [nlme]: OFDM option: 0
[INFO] [nctb]: Initialized CCA threshold: 35, -60, -60, -100
[INFO] [fhss]: fhss Configuration set, UC channel: 0, BC channel: 0, UC CF: 0, BC CF: 0, channels: BC 35 UC 1, uc dwell: 255, bc dwell: 255, bc interval: 0, psi: 0, ch retries: 0
[INFO] [wspc]: NW keys remove
[INFO] [wsbs]: Making parent selection in 401 s
[INFO] [wsbs]: Lancement du thread d'écoute RX
[INFO] [wsbs]: Send PAN advertisement Solicit
[TX Radio Bridge] Sent packet of 52 bytes
[RX Radio Bridge] Received packet of 114 bytes
[RX Radio] 114 octets | Type: BEACON (0x0000) | Sec:0 FP:0 AR:0 PIC:0 DM:0 SM:0
3 00 00 3f 46 e0 c4 37 22 12 c5 e1 4f 62 20
[RX Radio Bridge] Received packet of 61 bytes
[RX Radio] 61 octets | Type: BEACON (0x0000) | Sec:0 FP:0 AR:0 PIC:0 DM:0 SM:0
[TX Dump] [08 octets]: 00 00 01 e3 34 12 ed 64 f9 14 00 4b 12 00 0e 01 00 00 00 01 05 15 01 02 06 00 00 06 15 02 00 00 f0 0
0 00 01 01 00 22 00 05 04 00 00 00 23 0e
[INFO] [p1p6]: icmp !
[INFO] [p1p6]: bah 667
[INFO] [p1p6]: bah 437
[INFO] [p1p6]: bah 417
[INFO] [p1p6]: bah 1457
[INFO] [p1p6]: ipv6_stack called
[INFO] [p1p6]: icmp !
[INFO] [p1p6]: bah 667
[INFO] [p1p6]: bah 437
[INFO] [p1p6]: bah 417
[INFO] [p1p6]: bah 1457
[INFO] [wsbs]: Send PAN configuration
[TX Radio Bridge] Sent packet of 114 bytes
[INFO] [p1p6]: ipv6_stack called
[INFO] [p1p6]: icmp !
[INFO] [p1p6]: bah 667
[INFO] [p1p6]: bah 437
[INFO] [p1p6]: bah 417
[INFO] [p1p6]: bah 1457
[INFO] [p1p6]: ipv6_stack called
[INFO] [p1p6]: icmp !
[INFO] [p1p6]: bah 667
[INFO] [p1p6]: bah 437
[INFO] [p1p6]: bah 417
[INFO] [p1p6]: bah 1457
[INFO] [wsbs]: Send PAN advertisement
[TX Radio Bridge] Sent packet of 61 bytes
[INFO] [p1p6]: ipv6_stack called
[INFO] [p1p6]: icmp !
[INFO] [p1p6]: bah 667
[INFO] [p1p6]: bah 437
[INFO] [p1p6]: bah 417
[INFO] [p1p6]: bah 1457
```

Fig. 8. – Logs système : Validation de la transmission bidirectionnelle (TX/RX) du pont radio

Cette validation est très importantes : elle démontre que le blocage identifié ne provient pas de l'architecture de pontage, mais bien d'une défaillance au sein de la pile radio bas niveau. L'architecture développée est ainsi agnostique au support physique et prête à fonctionner de manière transparente dès qu'une version stable du driver radio sera déployée sur cette plateforme matérielle.

8.2.4 État d'avancement du pont direct (bridge_l2_direct.c)

Parallèlement aux validations effectuées, une variante optimisée, bridge_l2_direct.c, a été initiée. L'objectif de cette implémentation est de court-circuiter le passage par la pile réseau AF_PACKET afin de minimiser davantage la latence par un accès direct aux buffers du driver radio.

À ce jour, cette version demeure en phase d'investigation. L'intégration complète requiert une synchronisation plus fine avec les interruptions du contrôleur radio, ce qui n'a pas encore atteint l'état de stabilité fonctionnelle nécessaire pour une exploitation en production. Ce module demeure toutefois un axe de développement prioritaire, représentant la prochaine étape logique pour maximiser les performances de l'architecture une fois la plateforme matérielle totalement stabilisée.

8.2.5 Analyse de l'empreinte mémoire

Afin de mesurer la quantité de mémoire non volatile occupée par la compilation de la Nanostack et de Zephyr sur le microcontrôleur, une analyse de l'empreinte mémoire a été

réalisée. Le Tableau 4 détaille l'occupation mémoire pour les trois applications de test développées dans ce projet.

Nom	Flash	RAM
app_ethernet	355.728 Ko	74.256 Ko
app_border_router	516.796 Ko	120.4 Ko
app_device_node	513.14 Ko	120.336 Ko

Tableau 4. – Empreinte mémoire (Flash et RAM) après compilation

L'analyse de ces résultats met en évidence plusieurs aspects clés de l'intégration système :

- **Impact fonctionnel de la pile réseau** : L'application filaire (app_ethernet) présente l'empreinte la plus réduite avec environ 355 Ko de Flash et 74 Ko de RAM . En revanche, l'activation des fonctionnalités Wi-SUN complètes pour le nœud (app_device_node) et le routeur de bordure (app_border_router) entraîne une augmentation significative de l'occupation mémoire, atteignant environ 513 Ko à 516 Ko de Flash et 120 Ko de RAM. Cette différence de près de 160 Ko de Flash et 46 Ko de RAM s'explique par l'intégration de la pile de routage maillé (RPL, FHSS, Trickle timers), des services de découverte (PAN Discovery), ainsi que de la bibliothèque de sécurité **Mbed TLS** nécessaire pour l'authentification EAP-TLS de niveau 2.
- **Compatibilité avec la cible matérielle** : Le microcontrôleur CC1352P7 dispose de 704 Ko de Flash (ROM) et 144 Ko de RAM. Les résultats confirment que les deux applications Wi-SUN compilées sont pleinement compatibles avec les limites physiques de la cible. L'occupation de la Flash reste inférieure à 74% de la capacité totale, laissant une marge de sécurité confortable pour le code applicatif.
- **Marges de la mémoire vive (RAM)** : Avec une consommation de RAM s'élevant à environ 120 Ko (soit 83% de la RAM disponible), la marge de manœuvre restante pour l'exécution d'applications métiers complexes est plus restreinte. Ce constat valide a posteriori les choix d'implémentation stricts effectués lors du portage, notamment la mise en place d'un pool statique de contextes cryptographiques AES au lieu d'allocations dynamiques, afin de prévenir tout risque de fragmentation ou de dépassement de pile (*stack overflow*) en cours d'exécution.

9 Conclusion

9.1 Etat final du projet

Ce travail de Bachelor a permis de concrétiser le portage de la pile réseau **Nanostack** au sein de l'environnement **Zephyr**. Le projet a abouti à une architecture modulaire, capable de gérer des flux réseau de manière transparente via un pontage logiciel robuste.

Au terme de ce développement, les objectifs atteints démontrent la viabilité technique de l'approche adoptée :

- **Portage de la pile** : L'intégration de la **Nanostack** dans l'écosystème **Zephyr** est effective, offrant une base pérenne pour les applications IoT.
- **Communication filaire** : Le pont Ethernet est entièrement fonctionnel, permettant une communication IPv6 stable, incluant la gestion complète des couches L2 et L3.
- **Couche d'adaptation radio** : Le développement du pont radio a été validé au niveau structurel, confirmant la capacité du système à traiter les flux TX/RX en conditions réelles.

Certaines étapes, cependant, n'ont pu atteindre leur plein aboutissement en raison de limitations matérielles identifiées :

- **Stabilité de la pile radio Wi-SUN** : L'intégration complète du protocole Wi-SUN reste partielle. Bien que le pontage radio soit opérationnel, l'immaturité actuelle du pilote fourni pour le SoC CC1352 a empêché la finalisation d'un exemple fonctionnel complet.
- **Optimisation directe** : La variante optimisée `bridge_l2_direct.c`, bien qu'initée, nécessite des travaux complémentaires de synchronisation matérielle pour garantir sa fiabilité.

Ces éléments ne constituent pas une fin en soi, mais plutôt une base technique solide. Le pontage étant validé, la migration de la pile vers un pilote radio stabilisé ou une plateforme matérielle mieux supportée permettra de finaliser l'usage du Wi-SUN. Ce projet offre ainsi une architecture prête à l'emploi, dont la flexibilité facilitera les futures étapes de développement.

9.2 Intégration dans l'écosystème des sockets BSD de Zephyr

Parmi les objectifs optionnels définis dans le cahier des charges, l'intégration complète de la **Nanostack** au sein du sous-système réseau natif de **Zephyr** n'a pas pu être menée à son terme. L'accomplissement de cette tâche aurait permis aux applications d'interagir avec la pile Wi-SUN de manière transparente via l'API standardisée des sockets BSD de Zephyr.

Néanmoins, les travaux d'analyse menés au cours de ce projet confirment que ce rapprochement représente un défi architectural conséquent :

- **Le couplage des pilotes** : Comme mis en évidence lors de la conception du pont filaire, les pilotes de périphériques réseau de Zephyr sont intimement liés à ses couches L2 natives (comme `ETHERNET_L2` ou `IEEE802154_L2`). Réacheminer les paquets depuis les sockets BSD système vers la Nanostack nécessiterait de concevoir une couche de multiplexage ou d'exploiter le mécanisme d'offloading de sockets de Zephyr (*Socket Offloading*).
- **L'encapsulation des tampons** : L'interfaçage imposerait de traduire dynamiquement les descripteurs de paquets natifs de Zephyr (`net_pkt`) en structures de tampons internes de la Nanostack, ce qui induirait une surcharge de traitement et une complexité de gestion mémoire.

Cette intégration constitue ainsi un chantier de développement majeur à part entière. Elle figure en tête des perspectives futures pour ce portage, ouvrant la voie à une intégration native de la technologie Wi-SUN dans les prochaines versions du RTOS Zephyr.

9.3 Perspectives futures

Le travail réalisé dans le cadre de ce Travail de Bachelor pose les bases fonctionnelles du portage de la pile **Nanostack** sous **Zephyr RTOS**. Plusieurs axes d'amélioration et développements futurs permettraient de consolider ce projet et d'envisager son utilisation dans des déploiements industriels.

9.3.1 Caractérisation métrologique et énergétique

Bien que la latence et la stabilité aient été validées sur une liaison filaire point à point, il serait pertinent de mener des campagnes de mesures approfondies dans un environnement de réseau maillé multi-sauts :

- **Analyse de performance** : Évaluer le débit effectif, la perte de paquets et la latence lors de la reconstruction dynamique de routes par le protocole RPL.
- **Profilage de consommation** : Mesurer précisément (via un analyseur de puissance ou un outil de type **Power Profiler**) la signature énergétique du microcontrôleur CC1352P7

lors des phases de veille active (`K_FOREVER`) et de réveil radio, afin de certifier l'autonomie sur batterie des nœuds finaux.

9.3.2 Contribution et correction du pilote matériel Texas Instruments

La limitation actuelle obligeant à restreindre les communications au canal 0 découle d'un bug dans le calcul des fréquences au sein du pilote IEEE 802.15.4 de Zephyr. Une perspective clé consisterait à corriger cette fonction au niveau du pilote natif de Zephyr afin de prendre en charge l'ensemble des 35 canaux de la bande européenne. Cette contribution **upstream** permettrait de réactiver le saut de fréquence (FHSS) et de rendre le système pleinement conforme aux spécifications Wi-SUN FAN.

9.3.3 Intégration par déchargement de sockets (Socket Offloading)

À terme, l'intégration système idéale consisterait à implémenter la Nanostack comme pile IP principale pour l'interface radio via l'API de déchargement (**Socket Offloading**)[17] de Zephyr. Ce mécanisme permettrait aux applications d'utiliser les fonctions de sockets standards du système d'exploitation (`zsock_socket`, `zsock_sendto`, etc.) tout en bénéficiant de l'efficacité et de la sécurité de la Nanostack en arrière-plan, sans modification du code applicatif.

9.4 Planification finale

Ce chapitre présente le planning final et effectif des travaux réalisés tout au long de ce projet de Bachelor.

L'implémentation de la couche d'abstraction matérielle (HAL) ainsi que la phase de test et de validation ont requis un investissement en temps nettement supérieur aux estimations initiales. La complexité liée à l'écriture des ponts matériels a été sous estimée. Ce décalage a néanmoins été en partie compensé par la phase d'isolation métier/kernel et d'inventaire des fonctions à porter, qui s'est avérée plus rapide qu'escompté.

Cependant, cette redistribution de la charge horaire a impacté le périmètre final du projet : le temps restant n'a pas permis d'aborder l'intégration de la Nanostack au sein des sockets BSD natives de Zephyr, qui constituait un objectif initial optionnel. Les efforts se sont concentrés sur la robustesse du pontage de niveau 2 et de l'ordonnancement de la pile, garantissant ainsi la fiabilité des communications physiques.

CONCLUSION

Phase	Tâches associées	Temps alloué	Proportion
1. Base de code	• Fork de mbed-os et zephyr-os - Analyse de la hiérarchie de la Nanostack	20 h	4%
2. Prise en main Zephyr	• Installation de la toolchain - Compilation kernel + exemples - Programme de test	40 h	9%
3. Étude comparative	• Analyse de l'évent-loop - Interfaces et drivers réseau - Isolation des composants	50 h	11%
4. Rapport intermédiaire	• Rédaction de l'état d'avancement - Explication théorique	20 h	4%
5. Isolation métier/kernel	• Copie de la partie agnostique - Inventaire des fonctions à porter	10 h	2%
6. Création de la HAL	• Remplacement des appels kernel - Implémentation pour zephyr	180 h	40%

Phase	Tâches associées	Temps alloué	Proportion
7. Phase de tests	- Écriture des tests de validation - Dev d'un programme de communication P2P	95 h	21%
8. Intégration (Optionnel)	Portage pour appel via les sockets BSD	5 h	1%
9. Rapport final	- Documentation des implémentations - Bilan du projet	30 h	7%
Total effectif		450 h	100%

9.4.1 Diagramme de Gantt final

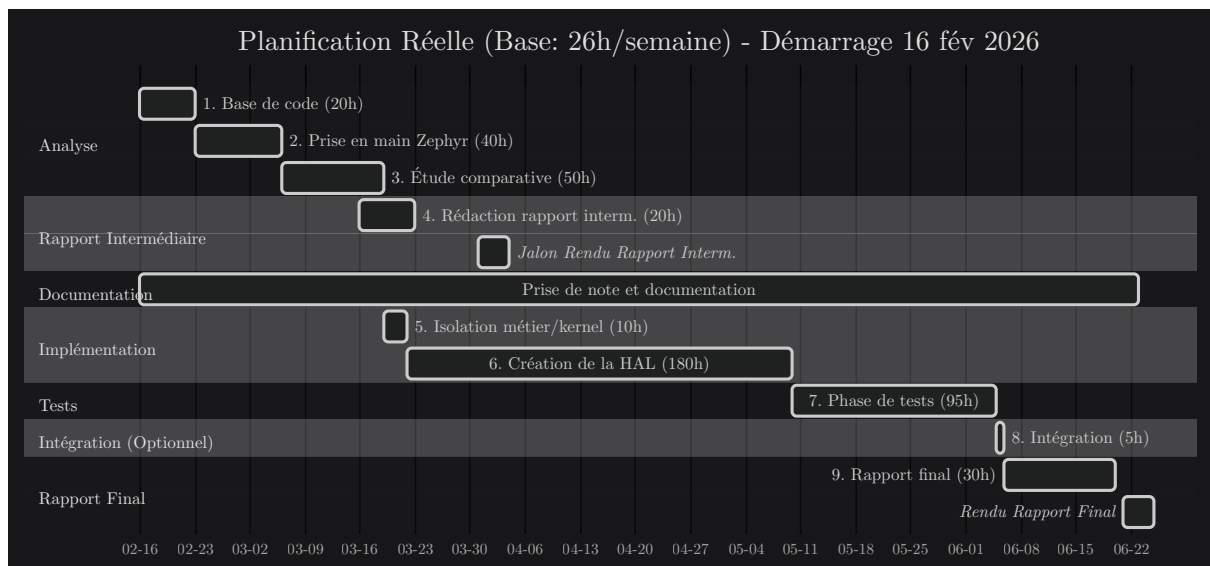


Fig. 9. – Diagramme de Gantt de la planification finale et effective

9.5 Conclusion personnelle

La réalisation de ce travail de Bachelor a constitué une expérience particulièrement enrichissante, tant sur le plan technique que méthodologique.

Ce projet m'a permis d'appréhender les subtilités et la rigueur indispensables aux travaux de portage de logiciels embarqués complexes. En me plongeant au cœur de la Nanostack, j'ai approfondi mes compétences en programmation réseau et affiné ma compréhension des mécanismes de transmission sans fil sub-GHz. Sur le plan de l'ingénierie système, l'opportunité de manipuler et de comparer des systèmes d'exploitation temps réel (RTOS) aux philosophies distinctes, tels que le moderne **Zephyr** et le plus historique **Mbed OS**, s'est révélée extrêmement formatrice. Le travail direct sur du matériel physique varié (modules de développement Texas Instruments et cartes Ethernet) m'a confronté aux réalités concrètes et parfois imprévues de l'intégration matérielle.

Enfin, ce travail a renforcé mes compétences en gestion de projet. Faire face à des imprévus techniques, évaluer l'avancement réel face aux objectifs et adapter la planification sous contrainte de temps m'ont appris à structurer et à prioriser les tâches de manière autonome. Ce projet représente ainsi un jalon clé dans mon parcours de futur ingénieur.

9.6 Remerciements

Je tiens à exprimer ma profonde gratitude aux personnes qui ont contribué à la réalisation de ce Travail de Bachelor.

Mes remerciements s'adressent tout d'abord au Professeur Favrat, pour la qualité de son encadrement et son accompagnement tout au long de ce projet.

Je souhaite également remercier chaleureusement Yann Charbon pour ses précieux conseils, sa disponibilité et la pertinence de ses réponses à mes nombreuses interrogations.

Enfin, je remercie Eva Ray, Thomas Germano, Melissa Gehring et Manuel Cabras pour leur relecture attentive de ce document et leurs suggestions constructives.

Benoît Delay

CONCLUSION

Bibliographie

- [1] « Wi-SUN : Les réseaux sans fil intelligents et omniprésents expliqués ». Consulté le: 1 avril 2026. [En ligne]. Disponible sur: <https://fr.digi.com/solutions/by-technology/wi-sun>
- [2] « Wi-SUN FAN Security Concepts and Design Considerations | Security Concepts and Design Considerations | Wi-SUN | v1.10.4 | Silicon Labs ». Consulté le: 1 avril 2026. [En ligne]. Disponible sur: <https://docs.silabs.com/wisun/1.10.4/wisun-security-concepts-design-considerations/>
- [3] « The Linux Foundation Announces Project to Build Real-Time Operating System for Internet of Things Devices | Zephyr Project ». Consulté le: 27 mars 2026. [En ligne]. Disponible sur: <https://web.archive.org/web/20160310073146/https://www.zephyrproject.org/news/linux-foundation-announces-project-build-real-time-operating-system-internet-things-devices>
- [4] « Mesh tutorial - API references and tutorials | Mbed OS 6 Documentation ». Consulté le: 1 avril 2026. [En ligne]. Disponible sur: <https://os.mbed.com/docs/mbed-os/v6.16/apis/connectivity-tutorials.html>
- [5] « Important Update on Mbed | Mbed ». Consulté le: 29 mars 2026. [En ligne]. Disponible sur: <https://os.mbed.com/blog/entry/Important-Update-on-Mbed/>
- [6] « CMSIS RTOS - Handbook | Mbed ». Consulté le: 1 avril 2026. [En ligne]. Disponible sur: <https://os.mbed.com/handbook/CMSIS-RTOS>
- [7] « West (Zephyr's meta-tool) — Zephyr Project Documentation ». Consulté le: 1 avril 2026. [En ligne]. Disponible sur: <https://docs.zephyrproject.org/latest/develop/west/index.html>

- [8] « System Power Management — Zephyr Project Documentation ». Consulté le: 1 avril 2026. [En ligne]. Disponible sur: <https://docs.zephyrproject.org/latest/services/pm/system.html>
- [9] « Device Power Management — Zephyr Project Documentation ». Consulté le: 1 avril 2026. [En ligne]. Disponible sur: <https://docs.zephyrproject.org/latest/services/pm/device.html>
- [10] « Important update on Mbed - End of Life - Mbed OS ». Consulté le: 29 mars 2026. [En ligne]. Disponible sur: <https://forums.mbed.com/t/important-update-on-mbed-end-of-life/23644>
- [11] « Modules (External projects) — Zephyr Project Documentation ». Consulté le: 29 mars 2026. [En ligne]. Disponible sur: <https://docs.zephyrproject.org/latest/develop/modules.html>
- [12] « Interrupts — Zephyr Project Documentation ». Consulté le: 23 juin 2026. [En ligne]. Disponible sur: <https://docs.zephyrproject.org/latest/kernel/services/interrupts.html>
- [13] « Timers — Zephyr Project Documentation ». Consulté le: 22 juin 2026. [En ligne]. Disponible sur: <https://docs.zephyrproject.org/latest/kernel/services/timing/timers.html>
- [14] « Random Number Generation — Zephyr Project Documentation ». Consulté le: 22 juin 2026. [En ligne]. Disponible sur: <https://docs.zephyrproject.org/latest/services/crypto/random/index.html>
- [15] « L2 Layer Management — Zephyr Project Documentation ». Consulté le: 22 juin 2026. [En ligne]. Disponible sur: https://docs.zephyrproject.org/latest/services/connectivity/networking/api/net_l2.html
- [16] « Threads — Zephyr Project Documentation ». Consulté le: 23 juin 2026. [En ligne]. Disponible sur: <https://docs.zephyrproject.org/latest/kernel/services/interrupts.html>
- [17] « Network Traffic Offloading — Zephyr Project Documentation ». Consulté le: 22 juin 2026. [En ligne]. Disponible sur: https://docs.zephyrproject.org/latest/services/connectivity/networking/api/net_offload.html#id5

Figures

Fig. 1	Planification Initiale	25
Fig. 2	Transaction driver -> Nanostack	33
Fig. 3	Architecture du mécanisme de pontage réseau	49
Fig. 4	Flux de configuration des ponts réseaux	50
Fig. 5	Architecture et flux de données à travers le pont logiciel bridge_l2.c	52
Fig. 6	Processus de découverte de voisins (NDP) à travers le pont logiciel	62
Fig. 7	Analyse des échanges ICMPv6 : preuve de la bidirectionnalité du pont	63
Fig. 8	Logs système : Validation de la transmission bidirectionnelle (TX/RX) du pont radio	68
Fig. 9	Diagramme de Gantt de la planification finale et effective	75

FIGURES

Tables

Tableau 1	Glossaire des termes techniques	15
Tableau 2	Comparaison des méthodes d'intégration logicielle sous Zephyr	36
Tableau 3	Analyse des performances de latence ICMPv6	65
Tableau 4	Empreinte mémoire (Flash et RAM) après compilation	69

Liste des extraits de code

Liste 1	Arborescence type du module Zephyr pour la Nanostack	37
Listing 2	Déclaration du module dans le west.yml de l'application	37
Listing 3	Fichier Kconfig à la racine du module	38
Listing 4	Structure du fichier Kconfig	42
Listing 5	Compilation conditionnelle	42
Listing 6	Liaison dynamique des dépendances	43
Listing 7	Gestion du conflit device via le fichier glue.h	43
Listing 8	Forcer l'inclusion de glue.h via CMake	44
Listing 9	Gestion des sections critiques	44
Listing 10	Mapping des timers	45
Listing 11	Abstraction de l'aléa	45
Listing 12	Sélection de Mbed TLS via Kconfig	46
Listing 13	Gestion statique et réentrante des contextes de chiffrement AES	47
Listing 14	Structure net_if_dev de Zephyr	51
Listing 15	Macro d'initialisation d'une interface Ethernet dans Zephyr	51
Listing 16	Virtualisation de l'adresse MAC via le bit d'administration locale	53
Listing 17	Initialisation et séparation des sockets TX et RX	54
Listing 18	Thread de réception radio asynchrone et filtrage	55
Listing 19	Limitation des canaux dans le pilote de périphérique TI CC1352 de Zephyr . . .	56
Listing 20	Boucle d'événements et thread dédié à la Nanostack	59
Listing 21	Gestion de la connexion TCP et envoi du compteur dans le Tasklet	64
Listing 22	Configuration Wi-SUN et restriction de canal sur le nœud périphérique	67